

Article

A GraphRAG-Based Dual-Path Structural Diffusion Retrieval Framework for Requirement-Code Traceability Link Recovery

Zhiqi Zhao  and Xiaoliang Xu * 

School of Computer Science and Technology, Hangzhou Dianzi University, No. 1158, 2nd Avenue, Qiantang District, Hangzhou 310018, China; 231050079@hdu.edu.cn

* Correspondence: xxl@hdu.edu.cn

Abstract

Requirement-Code Traceability Link Recovery (RC-TLR) maps requirements to the code artifacts that implement them. The task remains difficult because requirements and source code use different vocabularies and expose different structural cues. Recent large language model (LLM) methods can improve trace-link classification, but the final decision still depends on which candidate artifacts reach the validator. We propose DSDR, a Graph Retrieval-Augmented Generation (GraphRAG)-based dual-path structural diffusion framework for zero-shot RC-TLR. DSDR uses pre-trained embedding models and LLMs without project-specific trace supervision. It builds three-layer heterogeneous graphs for requirements and code, links requirement entities to code identifiers through cross-modal semantic alignment, and ranks candidate classes with semantic-gated forward diffusion and backward verification. The same diffusion states are backtracked into evidence paths for LLM-based structural validation. On five benchmark datasets, retrieval-only DSDR raises recall over the strongest evaluated RAG-style retrieval baseline on all five tasks and gives the top retrieval-only result on four tasks in this comparison. With GPT-4o validation, DSDR yields a trace-link set with higher precision than the strongest evaluated LLM-based baseline in the same evaluation setup, while the recall-precision balance varies by project. The comparison suggests that structural evidence is useful for zero-shot RC-TLR retrieval and for precision-oriented validation.

Keywords: traceability link recovery; retrieval-augmented generation; information retrieval; large language models; software engineering

1. Introduction

Modern software changes through requirements, architecture, and code, often at different speeds. Requirement-Code Traceability Link Recovery (RC-TLR) keeps these layers connected by recovering links between requirements and code artifacts [1–3]. Accurate trace links help developers assess change impact, audit compliance, localize affected code, and maintain software quality over time [4–7].

The difficulty is that requirements and code describe the same system from different viewpoints [8]. Requirements are written in natural language for stakeholders; they usually describe goals, constraints, and domain concepts. Code is organized around identifiers, methods, classes, and control or dependency relations. This mismatch creates a cross-modal semantic gap between requirement text and code artifacts [9–11]. A requirement about password reset, for example, may not contain any token that appears as a code identifier. The implementation may instead involve token generation, expiration checks, database



Academic Editor: Tudor Groza

Received: 8 May 2026

Revised: 22 May 2026

Accepted: 26 May 2026

Published: 1 June 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

updates, and notification logic. Trace links are also often one-to-many: one requirement can be implemented by several methods or classes spread across the codebase [12–14]. The reverse pattern is common as well. Utility or infrastructure classes may support many requirements and behave as hub artifacts, which can dominate retrieval scores without giving requirement-specific evidence [15].

Existing RC-TLR methods address parts of this problem. Classical Information Retrieval (IR) and semantic-enhancement methods improve textual matching, but they still miss implementations that are reachable through dependencies rather than lexical overlap. Supervised neural and graph-learning methods can model richer artifact relations, but they require labeled trace links or project-specific training, and their predictions can be hard to inspect in certification-oriented settings [4,16,17]. Recent studies show that RC-TLR is moving from lexical IR toward neural, graph-based, and LLM/RAG-based pipelines [14,17–20]. However, this rapid progress has not removed several practical challenges. High-quality trace-link labels remain expensive, which limits the direct use of supervised learning methods in newly adopted projects. Requirement–code vocabulary mismatch also remains difficult when relevant implementations are expressed through identifiers, dependencies, and architectural relations rather than shared terms. In addition, retrieve-then-validate pipelines still depend strongly on candidate generation: an LLM validator can only judge code artifacts that have already been retrieved into its context. These challenges motivate a GraphRAG-based framework that integrates semantic alignment, structural diffusion, and evidence-guided validation for RC-TLR.

We evaluate DSDR under zero-shot RC-TLR. At inference time, DSDR observes only requirement and code contents; trace-link labels are held out for evaluation. This setting reflects a practical constraint in traceability work: high-quality trace-link annotation is expensive and usually requires domain expertise [21,22]. The model must map natural-language requirements to code structure from unlabeled project artifacts and general-purpose pre-trained models.

We study whether a GraphRAG-style pipeline can use requirement–code structure as the retrieval substrate instead of relying on text fragments alone. This study is guided by four research questions: **RQ1**: Can dual-path structural diffusion improve candidate exposure before LLM-based validation? **RQ2**: How do semantic gating, forward diffusion, backward verification, PPR propagation, and score normalization contribute to retrieval quality? **RQ3**: How sensitive is the framework to soft-alignment and diffusion parameters under a fixed evaluation setting? **RQ4**: How does evidence-guided LLM validation affect the final precision–recall trade-off compared with a RAG baseline?

We propose DSDR, a **D**ual-path **S**tructural **D**iffusion **R**etrieval framework for zero-shot RC-TLR. The novel contribution of DSDR is the integration of three-layer requirement and code graphs, cross-modal soft semantic alignment, semantic-gated dual-path structural diffusion, and path-based evidence construction for LLM validation. DSDR builds hierarchical heterogeneous graphs for requirements and code, aligns requirement entities with code identifiers through a semantic bridge, and runs semantic-gated diffusion in two directions. Forward diffusion searches from a requirement toward candidate classes. Backward verification computes how strongly each candidate class supports the requirement side. The diffusion states are then backtracked into evidence paths for LLM-based structural validation, so the validator receives graph evidence rather than retrieved text alone.

Our main contributions are as follows:

- We introduce DSDR, a GraphRAG-based framework for zero-shot RC-TLR that combines three-layer heterogeneous graph modeling, semantic-gated dual-path structural retrieval, and evidence-guided LLM-based structural validation. With GPT-4o valida-

tion, DSDR returns a trace-link set with higher precision than the strongest evaluated LLM-based baseline in this setting.

- We construct aligned three-layer representations for requirements and code. The requirement side follows a requirement–sentence–entity hierarchy, the code side follows a class–method–identifier hierarchy, and a weighted semantic bridge connects requirement entities to code identifiers.
- We design a dual-path structural diffusion retriever that combines forward candidate discovery with backward requirement support. Score normalization and semantic gating reduce hub-artifact effects while retaining structurally relevant candidates with weak surface similarity.

2. Related Work

2.1. IR-Based TLR

Early traceability link recovery work often casts TLR as an Information Retrieval (IR) task. Representative methods use the Vector Space Model, Latent Semantic Indexing, Latent Dirichlet Allocation, and related probabilistic or vector-space models to rank candidate artifacts by textual similarity to a query artifact [6,8,10,15,23–25]. Traditional traceability establishment also includes manually maintained trace matrices, rule- or model-based link creation, change-impact links across heterogeneous software artifacts, and visualization support for managing trace links in continuous development settings [26]. These methods are efficient, transparent, and still widely used as TLR baselines. Their main weakness is vocabulary mismatch: requirements and code artifacts often describe the same concept using different words, abbreviations, or abstraction levels, so lexical similarity alone can miss relevant code artifacts [27].

Later IR-based work refines artifact representation and matching granularity. Gao et al. [28] use biterms, namely co-occurring term pairs mined from requirement–code structures, to strengthen lexical and contextual evidence. Gao et al. [14] exploit intermediate artifacts to infer transitive trace links and combine them with biterm-based enhancement. Hey et al. [13] show that sentence-to-method matching narrows the granularity gap between whole requirements and whole classes. These methods improve the textual signal, but structural dependencies still play a secondary role rather than acting as the retrieval substrate.

2.2. Learning-Based TLR

Learning-based TLR methods replace hand-crafted similarity functions with representations or classifiers learned from software artifacts. Pre-trained code models are often used because they encode regularities from both natural language and programming languages. Lin et al. [16], for example, fine-tuned CodeBERT [29] to recover trace links between issues and commits. These models can capture richer semantics than lexical matching, but they depend on labeled links and on the transferability of repository-specific patterns.

Software structure has also been used to compensate for text-only matching. Static call graphs, dependency paths, and intermediate artifacts can expose relations that artifact text does not show, and earlier studies used these signals to infer implicit links or propagate relevance across related artifacts [30–33]. More recent work incorporates these relations into heterogeneous graph learning. Wang et al. [17] proposed HGNNLINK, which represents artifacts with embeddings, encodes IR-based similarities and code dependencies as typed graph edges, and applies heterogeneous message passing for link prediction. Its reported gains over GA-XWCoDe [34] indicate that semantic features and structural dependencies carry complementary evidence for RC-TLR.

This line of work points to a need for both semantic and structural representations. Supervised graph-learning methods, however, still require labeled trace links or project-

specific training procedures, and learned predictions do not always yield inspectable requirement–code evidence paths. DSDR keeps the structural view but changes the operating mode: it uses pre-trained semantic models and deterministic structural diffusion in a zero-shot RC-TLR setting, while preserving diffusion states that can be backtracked into explicit evidence paths.

2.3. LLM-Based TLR

Recent traceability studies use large language models as classifiers rather than treating them only as embedding generators. In prompt-based settings, the model receives a pair of artifacts and predicts whether a trace link exists. Rodriguez et al. [35] studied prompt strategies for traceability and found that Chain-of-Thought prompting [36] can improve reasoning behavior compared with direct classification prompts. Ali et al. [18] further argued that many earlier semantic methods still rely on shallow similarity, leaving much of the requirement–code semantic gap unresolved. Direct prompting does not scale well, because realistic artifact collections quickly exceed practical context windows and raise inference cost.

Retrieval-Augmented Generation (RAG) makes LLM use in TLR more practical by separating candidate retrieval from final validation [37]. Fuchß et al. [19] proposed LiSSA, a general-purpose RAG framework that retrieves candidate artifacts before asking an LLM to judge the retrieved pairs. Hey et al. [20] applied a related RAG-style pipeline to inter-requirement traceability and examined the role of CoT reasoning within the retrieve-then-generate workflow.

Current RAG-style TLR pipelines that use LLM validators still depend heavily on the retriever. The retrieved context may contain semantically similar snippets while omitting the call, containment, or bridge relations that explain why a code artifact implements a requirement. Hub-like artifacts can also enter the prompt and lead to explanations that are plausible but weakly tied to requirement-specific structure. In DSDR, candidate classes are ranked through semantic-gated diffusion over requirement and code graphs, and the validator receives evidence paths extracted from the ranking traces.

3. Materials and Methods

We define DSDR, a **D**ual-path **S**tructural **D**iffusion **R**etrieval framework with GraphRAG for RC-TLR. Given the requirements $\mathcal{R}$ and code classes $\mathcal{C}$, the task is to recover a trace-link set $\mathcal{T} \subseteq \mathcal{R} \times \mathcal{C}$. DSDR follows a retrieve-then-validate workflow: for each requirement $r \in \mathcal{R}$, it ranks candidate classes with graph-based structural evidence and submits only the top K candidate pairs to an LLM validator. Figure 1 summarizes the data flow.

DSDR has three stages organized around the intermediate artifacts passed between them. Stage I constructs requirement-side and code-side heterogeneous graphs and connects them through a semantic bridge. Stage II consumes the unified graph and bridge scores to run semantic-gated dual-path structural diffusion, producing ranked candidate classes and diffusion states. We refer to the complete three-stage framework as DSDR and to the Stage II retriever as Semantic-gated Dual-path Structural Diffusion Retrieval (SG-DSDR). Stage III reuses the diffusion states to extract high-contribution evidence paths and grounds LLM-based structural validation on those paths. This organization separates graph construction, retrieval, and validation, so the experiments report retrieval-only behavior, component ablations, parameter sensitivity, and evidence-path validation behavior separately.

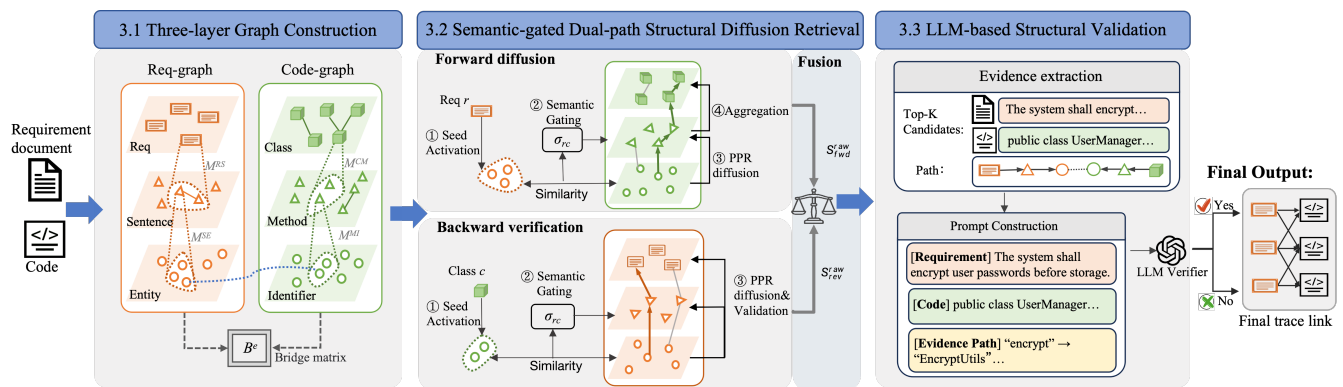


Figure 1. Workflow of Dual-path Structural Diffusion Retrieval (DSDR). Stage I builds requirement/code three-layer graphs and the cross-modal semantic bridge. Stage II performs semantic-gated forward/backward structural diffusion to rank Top-K candidate classes. Stage III extracts diffusion-supported evidence paths and uses them for Large Language Model (LLM)-based structural validation.

Zero-shot RC-TLR denotes evaluation with held-out benchmark trace links. General-purpose pre-trained encoders and LLMs are allowed, but benchmark labels are used only for evaluation, not for model construction, hyperparameter selection, or calibration.

3.1. Three-Layer Graph Construction

RC-TLR starts with a representational mismatch. Requirements express business intent in natural language, whereas source code realizes that intent through identifiers, containment relations, and static dependencies. Stage I converts both sides into layered graphs. Retrieval can then propagate evidence across comparable node types instead of matching whole artifacts as flat text.

3.1.1. Requirement-Side Three-Layer Graph $\mathcal{G}_R$

Given a collection of requirement documents $\mathcal{D}$, we represent the requirement side as a heterogeneous graph $\mathcal{G}_R = (V_R, \mathcal{A}_R)$ with three hierarchical layers.

The node set V_R has three layers. The **requirement layer** is $\mathcal{R} = \{r_i\}_{i=1}^{|\mathcal{R}|}$, where each r_i is an individual requirement entry. The **sentence layer** is $\mathcal{S} = \{s_j\}_{j=1}^{|\mathcal{S}|}$, obtained by sentence segmentation. The **entity/phrase layer** is $\mathcal{E} = \{e_k\}_{k=1}^{|\mathcal{E}|}$, where each e_k is a salient noun phrase or named entity extracted from a sentence using spaCy. The full node set is $V_R = \mathcal{R} \cup \mathcal{S} \cup \mathcal{E}$.

We use two sparse binary association matrices to encode cross-layer containment. The matrix $\mathbf{W}^{RS} \in \{0, 1\}^{|\mathcal{R}| \times |\mathcal{S}|}$ records requirement–sentence membership: $\mathbf{W}_{ij}^{RS} = 1$ when sentence s_j belongs to requirement r_i , and 0 otherwise. The matrix $\mathbf{W}^{SE} \in \{0, 1\}^{|\mathcal{S}| \times |\mathcal{E}|}$ records sentence–entity membership: $\mathbf{W}_{jk}^{SE} = 1$ when entity or phrase e_k appears in sentence s_j . The non-zero entries of these matrices define the requirement-side edge sets $\mathcal{L}_{RS}$ and $\mathcal{L}_{SE}$. We set $\mathcal{A}_R = \mathcal{L}_{RS} \cup \mathcal{L}_{SE}$, yielding $\mathcal{G}_R = (V_R, \mathcal{A}_R)$.

3.1.2. Code-Side Three-Layer Graph $\mathcal{G}_C$

The code-side graph contains the implementation units that can be linked to requirements and the local structures through which implementation intent is distributed.

We define the code-side graph as $\mathcal{G}_C = (V_C, \mathcal{L}_C)$. The node set V_C has three layers: a **class layer** $\mathcal{C} = \{c_u\}_{u=1}^{|\mathcal{C}|}$, where each c_u is a candidate code class; a **method layer** $\mathcal{M} = \{m_v\}_{v=1}^{|\mathcal{M}|}$, where each m_v is a concrete method body defined in a class; and an **identifier layer** $\mathcal{I} = \{i_w\}_{w=1}^{|\mathcal{I}|}$, where each i_w is a normalized identifier extracted from

method-level source code, including variable names, method/function names, and class names. We set $V_C = \mathcal{C} \cup \mathcal{M} \cup \mathcal{I}$.

Containment relations are encoded using sparse binary association matrices $\mathbf{W}^{CM} \in \{0, 1\}^{|\mathcal{C}| \times |\mathcal{M}|}$ and $\mathbf{W}^{MI} \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{I}|}$. The entry $\mathbf{W}_{uv}^{CM}$ is one when method m_v is defined in class c_u , and $\mathbf{W}_{vw}^{MI}$ is one when identifier i_w appears in method m_v . Lightweight static parsing with *tree-sitter* extracts explicit source-level dependencies. We store static calls in $\mathbf{W}^{call} \in \{0, 1\}^{|\mathcal{M}| \times |\mathcal{M}|}$, where an entry indicates that method m_v statically calls method $m_{v'}$, and inheritance or interface-implementation relations in $\mathbf{W}^{inh} \in \{0, 1\}^{|\mathcal{C}| \times |\mathcal{C}|}$.

The non-zero entries of these matrices define the code-side edge set. We define $\mathcal{L}_{CM} = \{(c_u, m_v) \mid \mathbf{W}_{uv}^{CM} = 1\}$, $\mathcal{L}_{MI} = \{(m_v, i_w) \mid \mathbf{W}_{vw}^{MI} = 1\}$, $\mathcal{L}_{call} = \{(m_v, m_{v'}) \mid \mathbf{W}_{vv'}^{call} = 1\}$, and $\mathcal{L}_{inh} = \{(c_u, c_{u'}) \mid \mathbf{W}_{uu'}^{inh} = 1\}$, and set $\mathcal{L}_C = \mathcal{L}_{CM} \cup \mathcal{L}_{MI} \cup \mathcal{L}_{call} \cup \mathcal{L}_{inh}$. This representation intentionally covers dependencies visible from source syntax. We currently use this static graph because it provides a fixed and reproducible analysis boundary across benchmark datasets, which supports stable comparison under the same retrieval setting. Dynamic dependency information is not incorporated in the present study. We therefore describe dynamic dependency modeling as future work rather than drawing conclusions about its concrete effects in our experiments.

3.1.3. Cross-Modal Soft Alignment via a Bridging Matrix

To connect the two graphs, we add cross-modal semantic edges between requirement entities and code identifiers. The bridge covers cases where a requirement term and a code identifier express the same concept without sharing surface tokens. For example, a requirement entity such as *bill* may correspond to an identifier such as *Invoice*. We construct a soft-alignment bridging matrix $\mathbf{B}$ using the pre-trained text embedding model specified in Section 3.4.3; non-zero entries of $\mathbf{B}$ become weighted virtual edges between $\mathcal{E}$ and $\mathcal{I}$.

For each entity–identifier pair, let $\mathbf{v}(e_k)$ denote the embedding of a requirement entity $e_k \in \mathcal{E}$ and $\mathbf{v}(i_w)$ denote the embedding of a code identifier $i_w \in \mathcal{I}$. We compute cosine similarity as:

$$\text{sim}(e_k, i_w) = \frac{\mathbf{v}(e_k)^\top \mathbf{v}(i_w)}{\|\mathbf{v}(e_k)\|_2 \cdot \|\mathbf{v}(i_w)\|_2}. \quad (1)$$

Before embedding code identifiers, we split camel-case and snake-case names into sub-tokens and embed the normalized identifier phrase. This preprocessing prevents compound identifiers from being treated as opaque strings.

To preserve graph sparsity and suppress noisy matches, we apply a similarity threshold τ . The bridging matrix $\mathbf{B} \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{I}|}$ is defined as:

$$\mathbf{B}_{kw} = \begin{cases} \text{sim}(e_k, i_w), & \text{if } \text{sim}(e_k, i_w) \geq \tau, \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

The threshold keeps weak entity–identifier pairs out of later propagation. We use τ as a coverage–sparsity operating parameter, not as a learned classifier threshold. The main experiments use $\tau = 0.1$ as a coverage-oriented bridge setting that preserves weak entity–identifier bridges for subsequent semantic gating. The diagnostic sensitivity study in Appendix A shows that $\tau = 0.1$ and $\tau = 0.2$ form the strongest low-threshold region, while higher thresholds remove too many entity–identifier links. A lower threshold therefore increases the initial bridge coverage, but it does not make all additional matches equally influential: the semantic gate in the subsequent diffusion step attenuates weakly requirement-related activations and lets useful candidates retain more probability mass. Thus, τ should be interpreted as a coverage–sparsity trade-off parameter. In deployment,

τ could be selected by retaining an upper quantile of entity–identifier similarities or by choosing the smallest threshold that preserves a target bridge density under the available retrieval budget.

After bridge construction, the unified retrieval graph is $\mathcal{G} = (V, \mathcal{L})$, where $V = V_{\mathcal{R}} \cup V_{\mathcal{C}}$ and $\mathcal{L} = \mathcal{L}_{\mathcal{R}} \cup \mathcal{L}_{\mathcal{C}} \cup \mathcal{L}_B$ with $\mathcal{L}_B = \{(e_k, i_w) \mid \mathbf{B}_{kw} > 0\}$. Bridge edges in $\mathcal{L}_B$ are weighted by $\mathbf{B}_{kw}$, whereas containment, call, and inheritance edges are unweighted unless otherwise stated.

3.2. Semantic-Gated Dual-Path Structural Diffusion Retrieval

Text-only retrieval can miss relevant code when requirements and code use different vocabulary, or when the relevant code is reachable mainly through dependencies. Prior work reports similar benefits from structural signals: they can improve candidate quality and recover links missed by semantic matching [17,30,31]. SG-DSDR keeps the propagation space deliberately narrow. It uses the containment-based seed projection and the method-call backbone recovered in Stage I. Inheritance edges remain in the constructed code graph as auxiliary structural context, but the equations below do not use them as a separate propagation channel.

Semantic similarity acts as a prior on structural propagation. Cross-modal bridges map requirement entities to code identifiers, while a semantic gate $g(r, c)$ reduces propagation toward classes that are weakly related to the current requirement. The retriever, SG-DSDR, computes two complementary scores: forward diffusion discovers candidate classes from a requirement, and backward verification computes how strongly a class supports the requirement side. The two scores are fused after normalization.

3.2.1. Semantic Gating and Gated PPR

For each requirement $r \in \mathcal{R}$ and class $c \in \mathcal{C}$, we precompute embeddings $\mathbf{e}_r = f(\text{text}(r)) \in \mathbb{R}^D$ and $\mathbf{e}_c = f(\text{text}(c)) \in \mathbb{R}^D$. We define a non-negative cosine gate as

$$g(r, c) = \max\left(0, \frac{\mathbf{e}_r^\top \mathbf{e}_c}{\|\mathbf{e}_r\|_2 \|\mathbf{e}_c\|_2}\right) \in [0, 1]. \quad (3)$$

Here $f(\cdot)$ denotes the embedding function, $\text{text}(r)$ is the original requirement text, and $\text{text}(c)$ is a class-level textual representation obtained by concatenating the class name, method/function names, and normalized identifier subtokens extracted from the class.

For a fixed requirement r , we collect class-wise gates into $\mathbf{g}_r^C \in [0, 1]^{|\mathcal{C}|}$ with $[\mathbf{g}_r^C]_c = g(r, c)$, and broadcast them to the method layer via class–method membership: $\mathbf{g}_r^M = (\mathbf{W}^{CM})^\top \mathbf{g}_r^C \in [0, 1]^{|\mathcal{M}|}$. Similarly, for a fixed class c , we define requirement-wise gates $\mathbf{g}_c^R \in [0, 1]^{|\mathcal{R}|}$ with $[\mathbf{g}_c^R]_r = g(r, c)$, and broadcast to sentence/entity layers by $\mathbf{g}_c^S = (\mathbf{W}^{RS})^\top \mathbf{g}_c^R$ and $[\mathbf{g}_c^E]_k = \frac{\sum_j \mathbf{W}_{jk}^{SE} \mathbf{g}_c^S[j]}{\sum_j \mathbf{W}_{jk}^{SE} + \epsilon}$.

Unlike the bridge threshold τ , the semantic gate is not binarized. It is a continuous attenuation factor: a low gate value does not delete a node from the graph, but it reduces the probability mass that the node can retain after each diffusion step. A structurally relevant class can remain reachable even when its surface similarity is weak. Globally connected classes lose mass unless they are also semantically close to the requirement.

For any non-negative adjacency matrix $\mathbf{A}$, we row-normalize it as $\text{RowNorm}(\mathbf{A}) = \mathbf{D}^{-1}\mathbf{A}$, where $\mathbf{D}_{ii} = \sum_j \mathbf{A}_{ij}$. In the transition matrices below, self-loops are added once before row normalization to avoid zero-degree rows; isolated methods or requirement nodes can then retain their own mass rather than producing undefined transitions. For the method call graph, we use an undirected caller–callee backbone $\tilde{\mathbf{A}}_{\text{call}} = \mathbb{I}[\mathbf{W}^{\text{call}} + (\mathbf{W}^{\text{call}})^\top > 0] + \mathbf{I}_{|\mathcal{M}|}$ and its row-stochastic transition

$\mathbf{P}_{\text{call}} = \text{RowNorm}(\tilde{\mathbf{A}}_{\text{call}})$. The symmetrization allows evidence to move along local call associations even when the requirement signal starts from a callee-side identifier; the later evidence-extraction step still reports paths in an explanatory order from requirement to candidate class. For the requirement-side three-layer graph, we form a symmetric adjacency on $V_{\mathcal{R}} = \mathcal{R} \cup \mathcal{S} \cup \mathcal{E}$ as the block matrix

$$\tilde{\mathbf{A}}_{\text{req}} = \begin{bmatrix} \mathbf{0} & \mathbf{W}^{RS} & \mathbf{0} \\ (\mathbf{W}^{RS})^\top & \mathbf{0} & \mathbf{W}^{SE} \\ \mathbf{0} & (\mathbf{W}^{SE})^\top & \mathbf{0} \end{bmatrix} + \mathbf{I}_{|V_{\mathcal{R}}|}, \quad \mathbf{P}_{\text{req}} = \text{RowNorm}(\tilde{\mathbf{A}}_{\text{req}}). \quad (4)$$

Let $\mathcal{N}(\mathbf{y}) = \mathbf{y}/\|\mathbf{y}\|_1$ if $\|\mathbf{y}\|_1 > 0$ and $\mathcal{N}(\mathbf{y}) = \mathbf{0}$ otherwise. Given a row-stochastic transition matrix $\mathbf{P}$, a personalization distribution $\mathbf{p}_0$, and a node-wise gate vector $\mathbf{g} \in [0, 1]^n$, we define *gated* PPR by iterating Equation (5) until convergence.

$$\mathbf{x}^{(t+1)} = \alpha \mathbf{p}_0 + (1 - \alpha) \mathbf{P}^\top \mathbf{x}^{(t)}, \quad \mathbf{x}^{(t+1)} \leftarrow \mathcal{N}(\mathbf{x}^{(t+1)} \odot \mathbf{g}). \quad (5)$$

If a projected seed vector is all zero before PPR, the corresponding directional raw score is defined as the zero vector and that direction contributes no evidence for the query or class. Otherwise, the iteration stops when the L_1 difference between consecutive vectors falls below 10^{-6} or after 100 iterations. We denote the resulting stationary scores as $\text{GPPR}(\mathbf{p}_0, \mathbf{P}, \alpha, \mathbf{g})$.

Computational footprint. For a project with N_r requirements, N_c candidate classes, N_e requirement entities, N_i code identifiers, N_m methods, and E_{call} call edges, the one-time semantic scoring stage embeds $N_r + N_c + N_e + N_i$ textual units. Bridge construction computes $N_e N_i$ entity–identifier cosine scores before thresholding, and semantic gating computes $N_r N_c$ requirement–class scores. After these project-level matrices are built, diffusion requires at most N_r forward GPPR runs on the method–call backbone and N_c backward-verification GPPR runs on the requirement graph, with each run bounded by 100 iterations. The resulting ranked lists are reused by both validator configurations. Thus, the validator stage scales as $N_r K$ candidate-pair calls, whereas retrieval-side graph construction and diffusion are deterministic project-level preprocessing steps. Appendix A reports cached-embedding time-to-retrieve measurements to make this computational footprint concrete.

3.2.2. Forward Diffusion for Candidate Discovery

Both directions in SG-DSDR follow the same computational pattern: construct directional seeds by matrix propagation, project them across modalities via $\mathbf{B}$, run gated PPR on the destination graph, and aggregate the diffusion scores to the destination layer. In the *forward* direction, the signal starts from a requirement and propagates on the method call backbone. The output is a class score vector for candidate discovery.

First, we activate requirement-side entity seeds. Let $\mathbf{a}_{\text{req}} \in \{0, 1\}^{|\mathcal{R}|}$ be the one-hot indicator of requirement r . We obtain entity seeds by matrix propagation on the requirement-side containment structure: $\mathbf{a}_{\text{sent}} = (\mathbf{W}^{RS})^\top \mathbf{a}_{\text{req}}$ and $\mathbf{a}_{\text{ent}} = (\mathbf{W}^{SE})^\top \mathbf{a}_{\text{sent}} \in \mathbb{R}^{|\mathcal{E}|}$.

Next, we project the entity seeds across the semantic bridge. Using $\mathbf{B} \in \mathbb{R}^{|\mathcal{E}| \times |\mathcal{I}|}$, we compute identifier seeds $\mathbf{a}_{\text{id}}^{(0)} = \mathbf{B}^\top \mathbf{a}_{\text{ent}} \in \mathbb{R}^{|\mathcal{I}|}$.

Given the projected identifiers, we run gated diffusion on the method call graph. We aggregate identifier seeds to methods by $\mathbf{s}_m = \mathbf{W}^{MI} \mathbf{a}_{\text{id}}^{(0)} \in \mathbb{R}^{|\mathcal{M}|}$ and form the gated personalization distribution $\mathbf{p}_0^{\text{fwd}} = \mathcal{N}(\mathbf{s}_m \odot \mathbf{g}_r^M)$. If $\|\mathbf{s}_m \odot \mathbf{g}_r^M\|_1 = 0$, we skip the forward PPR for r and set $\mathbf{S}_{\text{fwd}}^{\text{raw}}[r, :] = \mathbf{0}$. Otherwise, $\mathbf{p}_0^{\text{fwd}}$ is a valid personalization distribution. We then run gated PPR on the method call transition $\mathbf{P}_{\text{call}}$, where the gate is applied after each propagation step:

$$\mathbf{h}_m^r = \text{GPPR}(\mathbf{p}_0^{\text{fwd}}, \mathbf{P}_{\text{call}}, \alpha_{\text{fwd}}, \mathbf{g}_r^M). \quad (6)$$

After method-level gated propagation, we aggregate method scores to classes:

$$\mathbf{S}_{\text{fwd}}^{\text{raw}}[r, :] = \mathbf{W}^{\text{CM}} \mathbf{h}_m^r. \quad (7)$$

The aggregation matches the class-level labels used by the benchmarks and preserves the total method-level evidence assigned to a class. It can reward classes that contain several mutually supporting methods. It can also favor large classes when only a small subset of their methods is requirement-relevant. Semantic gating and later row-wise normalization reduce, but do not remove, this class-size effect. More selective alternatives, such as max pooling, top- m method pooling, or learned attention over methods, are natural extensions when method-level supervision or developer-facing localization is available.

3.2.3. Backward Verification for Requirement Support

The *backward-verification* direction swaps the propagation direction. Seeds originate from the code-side structure of a candidate class, are projected back to requirement entities via $\mathbf{B}$, and then diffuse over the requirement-side heterogeneous graph. The resulting score measures requirement-side support for class c . It is a ranking signal, not a proof of logical consistency.

First, we activate code-side identifier seeds from the candidate class. Let $\mathbf{a}_{\text{cls}} \in \{0, 1\}^{|C|}$ be the one-hot indicator of class c . We obtain identifier seeds by matrix propagation on the code-side containment structure: $\mathbf{a}_m = (\mathbf{W}^{\text{CM}})^{\top} \mathbf{a}_{\text{cls}}$ and $\mathbf{a}_{\text{id}} = (\mathbf{W}^{\text{MI}})^{\top} \mathbf{a}_m \in \mathbb{R}^{|I|}$.

Next, we project identifier seeds back to requirement entities by $\mathbf{a}_{\text{ent}}^{(0)} = \mathbf{B} \mathbf{a}_{\text{id}} \in \mathbb{R}^{|E|}$. To run diffusion on $V_{\mathcal{R}} = \mathcal{R} \cup \mathcal{S} \cup \mathcal{E}$, we place the personalization mass on entity nodes: $\mathbf{q}_0 = [\mathbf{0}_{|\mathcal{R}|}; \mathbf{0}_{|\mathcal{S}|}; \mathbf{a}_{\text{ent}}^{(0)}]$ and gate it by $\mathbf{p}_0^{\text{rev}} = \mathcal{N}(\mathbf{q}_0 \odot \mathbf{g}_c^V)$, where $\mathbf{g}_c^V = [\mathbf{g}_c^R; \mathbf{g}_c^S; \mathbf{g}_c^E]$. If $\|\mathbf{q}_0 \odot \mathbf{g}_c^V\|_1 = 0$, we skip the backward-verification PPR for c and set $\mathbf{S}_{\text{rev}}^{\text{raw}}[:, c] = \mathbf{0}$.

The projected requirement-side seeds then diffuse over the requirement heterogeneous graph. We run gated PPR on the requirement transition $\mathbf{P}_{\text{req}}$, with the gate re-applied after each propagation step: $\mathbf{h}_{\text{req}}^c = \text{GPPR}(\mathbf{p}_0^{\text{rev}}, \mathbf{P}_{\text{req}}, \alpha_{\text{rev}}, \mathbf{g}_c^V)$. Let $\mathbf{h}_s^c$ denote the subvector restricted to sentence nodes.

After requirement-side gated propagation, we aggregate sentence support to requirements:

$$\mathbf{S}_{\text{rev}}^{\text{raw}}[:, c] = \mathbf{W}^{\text{RS}} \mathbf{h}_s^c. \quad (8)$$

3.2.4. z-Score Normalization

Forward and backward-verification scores have different distributions because they normalize over different requirement and class axes. Before fusion, we apply standard z-score normalization with a small numerical constant ϵ in the denominator. For the forward retriever, raw scores are standardized within each requirement across candidate classes, producing $\mathbf{Z}_{\text{fwd}}$. For the backward-verification retriever, raw scores are standardized within each code class across requirements, producing $\mathbf{Z}_{\text{rev}}$. This keeps the two directional scores comparable before interpolation without introducing another learned calibration step.

The final structural score fuses the standardized scores as

$$\mathbf{S}_{\text{struct}} = \beta \mathbf{Z}_{\text{fwd}} + (1 - \beta) \mathbf{Z}_{\text{rev}}, \quad (9)$$

where $\beta \in [0, 1]$ controls the trade-off between forward candidate discovery and backward requirement support. For each requirement, code candidates are ranked by $\mathbf{S}_{\text{struct}}$ and the top K classes are passed to Stage III.

3.3. LLM-Based Structural Validation

Stage II produces a relevance-ranked list of the top K candidate classes. Because retrieval remains a ranking stage, some candidates may be structurally connected to a requirement without actually implementing it. Stage III uses an LLM as a validator, but the LLM is not asked to infer trace links from unstructured candidate text alone. Instead, it receives candidate code together with explicit evidence paths extracted from the same graph, transition matrices, and diffusion states that produced the candidate ranking.

For a retrieved pair (r, c) , Stage III reuses the unified graph $\mathcal{G}$ constructed in Stage I and the Stage II transition matrices $\mathbf{P}_{\text{call}}$ and $\mathbf{P}_{\text{req}}$; it does not construct a separate explanation graph. It also uses the following cached quantities from Stage II: the requirement-side entity seeds $\mathbf{a}_{\text{ent}}$, the method seeds $\mathbf{s}_m$, the forward method diffusion scores $\mathbf{h}_m^r$, the backward-verification requirement-side diffusion scores $\mathbf{h}_{\text{req}}^c$, and the fused candidate score $\mathbf{S}_{\text{struct}}[r, c]$. These quantities identify the nodes that contributed to the ranking of c for r . Evidence extraction uses them to backtrack from high-scoring methods in c to the requirement-side seeds that activated those methods, rather than searching arbitrary graph paths.

A complete evidence path is an ordered typed chain $p = (r, s, e, i, m_0, \dots, m_t, c)$. Here S_r and E_r are the sentences and entities contained in requirement r , and M_c is the method set of candidate class c . The path starts from the queried requirement, selects a contained sentence $s \in S_r$ and entity $e \in E_r$, crosses a non-zero semantic bridge $\mathbf{B}_{e,i} > 0$ to an identifier i , attaches that identifier to a seed method m_0 with $\mathbf{W}_{m_0,i}^{MI} = 1$, follows zero or more call-association hops to an endpoint method $m_t \in M_c$, and ends at the candidate class c . The path length is bounded by L_{max} . Arrows show explanatory order from requirement to candidate: containment associations are traversed in the direction needed to connect layers, while bridge and call-association edges retain the weights used by retrieval.

PPR aggregates many walks into a stationary score, so no single path fully explains why c is ranked highly for r . We therefore extract a small set of explanatory paths with high diffusion contribution. First, endpoint methods are selected from M_c according to their forward diffusion scores $\mathbf{h}_m^r$ because the forward class score in Equation (7) is exactly the sum of method scores in the candidate class. Second, call chains are backtracked through edges that carried high diffusion mass. For a fixed requirement r , the contribution share of a call edge $u \rightarrow v$ to an activated method v is computed from the stationary forward diffusion state:

$$\rho_r(u, v) = \frac{\mathbf{h}_m^r[u] \mathbf{P}_{\text{call}}[u, v] \mathbf{g}_r^M[v]}{\sum_{u' \in N^-(v)} \mathbf{h}_m^r[u'] \mathbf{P}_{\text{call}}[u', v] \mathbf{g}_r^M[v] + \epsilon'}, \quad (10)$$

where $N^-(v)$ is the set of incoming neighbors of v under the symmetrized call backbone used by $\mathbf{P}_{\text{call}}$. Similarly, the contribution of a requirement entity–identifier bridge to a seed method m is

$$\chi_r(e, i, m) = \mathbf{a}_{\text{ent}}[e] \mathbf{B}_{e,i} \mathbf{W}_{m,i}^{MI}. \quad (11)$$

Equations (10) and (11) tie evidence extraction to the same seed activation, bridge weights, semantic gate, and diffusion scores used for candidate ranking. A returned path is therefore tied to the diffusion process rather than to graph reachability alone.

After a valid path is assembled, Algorithm 1 scores it using normalized node activation $\phi(n)$ and edge confidence $\omega(\ell)$. The node activation comes from Stage II: requirement-side nodes use $\mathbf{h}_{\text{req}}^c$, identifier nodes use bridge activation $\psi(i)$ computed in Step 2 of the algorithm, method nodes use $\mathbf{h}_m^r$, and the queried requirement and candidate class are assigned activation one. The path score is

$$\text{score}(p) = \frac{1}{|p|} \sum_{n \in p} \phi(n) + \frac{1}{|E_p|} \sum_{\ell \in E_p} \omega(\ell) - \lambda_\ell |E_p|, \quad (12)$$

where E_p is the edge set of path p and λ_ℓ is a small non-negative penalty that keeps unnecessarily long explanations from outranking compact evidence chains. Requirement containment and final method–class containment edges use unit confidence because they only make the path complete. Bridge edges use their semantic weight from $\mathbf{B}$, call edges use the diffusion contribution share $\rho_r(u, v)$ from Equation (10), and the identifier–method seed attachment uses the normalized bridge-to-method contribution $\chi_r(e, i, m_0)$ from Equation (11). A path receives a high evidence score only when it both connects r to c and follows high-contribution edges that helped activate the candidate.

The returned set $\mathcal{P}_{rc}$ contains the top K_p complete paths under Equation (12). If the candidate was ranked mainly by diffuse structural mass but no bounded path satisfies the typed-chain constraints described above and the type automaton in Algorithm 1, the evidence set is empty and the validator receives no path evidence for that candidate.

Example. For a password-encryption requirement and candidate class `userManager`, the entity node `encrypt` may connect through $\mathbf{B}$ to identifier `EncryptUtils`. If that identifier appears in method `hashPassword` and the method is defined in the candidate class, one complete path is

$$\begin{aligned} r \rightarrow s_{\text{enc}} \rightarrow e_{\text{encrypt}} \rightarrow i_{\text{EncryptUtils}} \\ \rightarrow m_{\text{hashPassword}} \rightarrow c_{\text{userManager}}. \end{aligned}$$

The paths expose explicit associative chains between requirements and code and provide the validator with graph-based evidence for the candidate link.

The validation prompt uses the extracted evidence paths. When valid paths exist, i.e., $\mathcal{P}_{rc} \neq \emptyset$, the three highest-scoring paths are inserted into the evidence field of the prompt. The prompt supplies graph-supported associations between the requirement and candidate code, rather than relying only on broad semantic similarity. Figure 2 shows the fixed structural validation template used for all validator calls in the main DSDR pipeline.

Structural Validation Prompt Template
Determine if there is a traceability link between the following requirement and code. ##Input: (1) Requirement: {req_text} (2) Code: {code_text} (3) Structural Evidence: {evidence} Based on the requirement, the code, and any structural evidence provided, reason step by step whether the code implements or relates to the requirement. Then answer with yes or no enclosed in <trace>...</trace>. ##Output: <trace>yes</trace> or <trace>no</trace>

Figure 2. Prompt used for structural validation with requirement text, candidate code, and evidence paths.

The LLM then outputs a binary trace decision for each retrieved candidate pair. The fixed yes/no output format is used for deterministic parsing. The current prompt supplies the highest-scoring supportive paths for the queried pair, so Stage III remains a fixed-template validator grounded in retrieved supporting evidence.

We parse the first normalized yes or no label in the response, including labels enclosed in <trace> tags; invalid or empty outputs produce no recovered link for that candidate pair. The final recovered trace-link set is defined as

$$\mathcal{T}_{\text{final}} = \{(r, c) \mid c \in \text{TopK}(r), \text{Label}_{\text{LLM}}(r, c) = \text{"yes"}\}. \quad (13)$$

Here, $\text{TopK}(r)$ denotes the top K candidate classes returned for requirement r .

Algorithm 1: Extraction of the top K_p complete evidence paths

Input: candidate pair (r, c) ; unified graph $\mathcal{G}$ from Stage I; Stage II transition matrices $\mathbf{P}_{\text{call}}$ and $\mathbf{P}_{\text{req}}$; bridge matrix $\mathbf{B}$; entity seeds $\mathbf{a}_{\text{ent}}$; method seeds $\mathbf{s}_m$; forward method scores $\mathbf{h}_m^r$; backward-verification requirement-side scores $\mathbf{h}_{\text{req}}^c$; fused structural scores $\mathbf{S}_{\text{struct}}$; limits b_s, b_e, b_i, b_m ; call-chain beam width b_ρ ; maximum path length L_{max} ; number of returned paths K_p .

Output: The top K_p complete evidence paths $\mathcal{P}_{rc}$.

Step 1: Select activated requirement nodes. Select the sentences S_r and entities E_r contained in requirement r . If a requirement contains more nodes than the limits permit, keep the top- b_s sentences and top- b_e entities according to $\mathbf{h}_{\text{req}}^c$; otherwise keep all contained nodes.

Step 2: Cross the semantic bridge. For every $e \in E_r$, collect identifiers i with $\mathbf{B}_{e,i} > 0$. Retain the top- b_i identifiers by bridge activation $\psi(i) = \max_{e \in E_r} \bar{h}_{\text{req}}^c(e) \mathbf{B}_{e,i}$, where $\bar{h}_{\text{req}}^c(e)$ is the min-max normalized backward-verification score of entity e . Denote the retained identifier set as $I_{r,c}$.

Step 3: Select candidate-contributing endpoint methods. Let M_c be the methods contained in candidate class c . Select endpoint methods $M_c^* = \text{Top}_{b_m} \{m \in M_c \mid \mathbf{h}_m^r[m] > 0\}$ ranked by $\mathbf{h}_m^r[m]$. These are the methods that directly contribute to $\mathbf{S}_{\text{fwd}}^{\text{raw}}[r, c] = \sum_{m \in M_c} \mathbf{h}_m^r[m]$. If $M_c^* = \emptyset$, no forward structural path explains the candidate and the algorithm returns $\emptyset$ for this pair.

Step 4: Backtrack high-contribution call chains. Starting from each endpoint method $m_t \in M_c^*$, perform beam backtracking over incoming neighbors under the same $\mathbf{P}_{\text{call}}$ transition used by Stage II, using Equation (10). At each step, retain only the top- b_ρ incoming neighbors by $\rho_r(u, v)$, stop when a seed-supported method m_0 with $\mathbf{s}_m[m_0] > 0$ is reached, and discard chains longer than L_{max} . The procedure produces call-association chains $m_0 \rightarrow \dots \rightarrow m_t$ that follow the dominant diffusion flow into the candidate class.

Step 5: Attach bridge evidence and build complete paths. For each seed-supported method m_0 , choose the highest-contribution bridge triples (e, i, m_0) by Equation (11), with $e \in E_r$, $i \in I_{r,c}$, and $\mathbf{W}_{m_0,i}^{MI} = 1$. Prepend the corresponding requirement containment edge $r \rightarrow s \rightarrow e$, where s is a retained sentence containing e , prepend the bridge edge $e \rightarrow i$, append the method-class containment edge $m_t \rightarrow c$, and retain only simple paths satisfying the type automaton

$$R \rightarrow S \rightarrow E \rightarrow I \rightarrow M \rightarrow M^* \rightarrow C,$$

where M^* denotes zero or more method-call association hops from the same Stage II transition and C must be the candidate class c . Discard any path that lacks a bridge edge or never reaches an identifier/method node.

Step 6: Rank and return evidence. Score each valid path using Equation (12), break ties by shorter length and larger final candidate score $\mathbf{S}_{\text{struct}}[r, c]$, and return the top K_p paths as $\mathcal{P}_{rc}$. If no valid path remains, return $\mathcal{P}_{rc} = \emptyset$ and leave the LLM evidence field empty.

3.4. Experimental Setup

We evaluate DSDR on five standard TLR tasks and compare it with baselines used in recent traceability studies. The comparison includes traditional IR and machine-learning

approaches reported in the LISSA study [19]; LISSA itself is the main RAG-style reference point.

Experimental questions. Following the evaluation style of recent retrieve-then-validate TLR work, the experiments answer the research questions stated in the Introduction. The main results test whether structural diffusion improves candidate coverage before LLM-based structural validation and how graph-derived evidence affects end-to-end link decisions. The ablation study checks semantic gating, dual-path diffusion, PPR propagation, and z-score normalization. The sensitivity study varies β , α , and K in the main text and reports an additional threshold sweep for τ in Appendix A to characterize the coverage–sparsity behavior of soft alignment. The case study inspects one recovery case and one cautionary case, separating the helpful and harmful sides of structural reinforcement.

3.4.1. Datasets

For the experiments, we primarily use the datasets distributed in the replication package released by Hey et al. [13]. The core evaluation covers three widely used projects, SMOS, ETour, and ITRUST. The datasets were curated by the Center of Excellence for Software & Systems Traceability, CoEST, and they cover education, tourism, and healthcare. The three projects differ in vocabulary and implementation style, giving the evaluation some variation in both domain language and code organization.

To keep the comparison aligned with prior work such as LISSA and FTLR, we also include DRONOLGY from the aerospace domain. DRONOLGY [38] provides multiple natural-language artifact types, and we evaluate two standard settings by treating the entries denoted RE and design definitions denoted DD as requirement-side inputs for separate TLR tasks. This setup lets us test whether retrieval behavior changes with the requirement form while the code graph stays the same.

Table 1 reports the scale and linguistic characteristics of all datasets. All natural-language artifacts are in English except for SMOS, which contains Italian requirements and Italian code identifiers. For every dataset, we use the requirement–code trace links released with LISSA directly as the ground-truth annotations.

Table 1. Benchmark dataset statistics. TL denotes ground-truth trace links.

Dataset	Language	Programming	Req	Code	TL
SMOS	Italian	Java	67	100	1044
eTour	English	Java	58	116	308
iTrust	English	Java	131	226	286
Dronology (RE)	English	Java, Python	99	423	602
Dronology (DD)	English	Java, Python	211	423	740

Dataset analysis. The benchmark suite stresses the retriever in more than one way. SMOS is compact but link-dense, so precision matters because many classes are plausibly related to many requirements. ITRUST has more requirements and classes but fewer links, so candidate coverage and recall are useful complements to precision. The two DRONOLGY tasks share the same code graph while changing the requirement-side form from RE entries to DD entries. This mix reduces the chance that the reported trends come from one project size, natural language, or link-density regime.

3.4.2. Evaluation Metrics

We evaluate performance using standard metrics from prior TLR studies [4,5]: Precision, Recall, F_1 -score, and F_2 -score. Following the LISSA evaluation setup, metrics are

computed at the requirement level and then averaged over requirements. For each requirement r , let G_r be the released ground-truth code-class set and $\hat{G}_r$ be the recovered code-class set. We compute

$$P_r = \frac{|G_r \cap \hat{G}_r|}{|\hat{G}_r|}, \quad R_r = \frac{|G_r \cap \hat{G}_r|}{|G_r|}, \quad F_{1,r} = \frac{2P_r R_r}{P_r + R_r}, \quad F_{2,r} = \frac{5P_r R_r}{4P_r + R_r}. \quad (14)$$

When a denominator is zero, the corresponding requirement score is set to zero. The reported dataset-level values are macro-averages over the requirement scores, e.g., $P = |\mathcal{R}|^{-1} \sum_{r \in \mathcal{R}} P_r$, and analogously for R , F_1 , and F_2 . This requirement-level aggregation follows the LiSSA evaluation setup and keeps the comparison tied to recent traceability results.

3.4.3. Experimental Settings

We follow the LiSSA [19] evaluation setup where it defines the comparison surface: the released benchmark tasks, ground-truth trace links, class-level output granularity, standard TLR metrics, and Top-20 retrieved-candidate budget. We instantiate the retrieve-then-validate comparison with the same OpenAI model family used by LiSSA. Specifically, we use `text-embedding-3-large` as the embedding model, matching the LiSSA configuration and providing a shared semantic space for natural-language entities, abbreviated identifiers, and multilingual artifacts. Keeping the embedding model aligned with LiSSA avoids confounding the comparison: if different embedding models were used, changes in retrieval quality could reflect the semantic encoder rather than the proposed structural diffusion and evidence-construction components. For LLM-based structural validation and classification, we use GPT-4o as a higher-capacity validator and GPT-4o-mini as a lower-cost validator. To isolate the contribution of structural evidence paths under the lower-cost validator, we additionally report a GPT-4o-mini simple-prompt variant that uses the same retrieved Top-20 candidates but omits the evidence-path field from the validator prompt. This row is a diagnostic prompt-ablation comparison only and does not redefine the Stage III prompt template in Figure 2. All DSDR experiments follow the zero-shot RC-TLR setting defined in Section 3; the OPT baseline rows follow the LiSSA convention of reporting tuned-optimal reference settings for comparison.

Unless otherwise stated, the bridge threshold is fixed to $\tau = 0.1$, the PPR restart probabilities are fixed to $\alpha_{\text{fwd}} = \alpha_{\text{rev}} = 0.3$, the forward-backward fusion weight is fixed to $\beta = 0.5$, the retriever retains the first $K = 20$ ranked candidates, and the LLM receives the three highest-scoring complete paths. These values are fixed zero-shot settings rather than dataset-specific tuned parameters: the same parameter tuple is used for all datasets in the main experiments. $K = 20$ follows the Top-20 candidate budget used by LiSSA, $\beta = 0.5$ assigns equal weight to forward and backward evidence, $\alpha = 0.3$ is a moderate PPR restart value, and $\tau = 0.1$ is a coverage-oriented threshold for semantic bridges. The threshold sweep in Appendix A shows that $\tau = 0.1$ and $\tau = 0.2$ produce the strongest retrieval region because they retain more soft-alignment bridges, whereas higher thresholds reduce bridge coverage. The extra bridges introduced by lower thresholds are still filtered by the requirement-specific semantic gate before and during diffusion, so the threshold mainly changes the coverage available to the gate rather than directly accepting all weak matches as final candidates. We therefore treat τ as a coverage-sparsity operating parameter rather than as a tuned optimum. For evidence path extraction, we keep all sentences and entities in the queried requirement, retain at most 20 bridged identifiers and 50 activated methods per candidate pair, set the call-chain beam width to $b_\rho = 5$, use $L_{\text{max}} = 8$, and set the length penalty to $\lambda_\ell = 0.01$. These values are shared across datasets to avoid dataset-specific label tuning. The sensitivity study in Section 4.3 and Appendix A

is diagnostic only and is not used for dataset-specific tuning. Following the LiSSA reporting protocol, we use a two-sided Wilcoxon signed-rank test for F1/F2 scores at $\alpha = 0.05$ as a results-side significance check.

Validator cost. We report a representative API cost for one complete experimental pass over the five benchmark tasks. The tasks contain 566 requirements; with $K = 20$, one pass submits 11,320 candidate-pair validation calls for each validator model. Candidate-pair prompts vary with requirement length, code length, and the extracted evidence paths; in our path-prompt runs, their input sizes range from about 3000 to 15,000 tokens. Using the listed OpenAI standard text-token prices for GPT-4o (\$2.50 input and \$10.00 output per 1M tokens) and GPT-4o-mini (\$0.15 input and \$0.60 output per 1M tokens) (<https://platform.openai.com/docs/pricing/>, accessed on 1 January 2026), and using a 10,000-token representative input estimate per validation call with a short classification-output estimate of 20 tokens, the one-pass validator-token cost is \$285 for GPT-4o and \$17 for GPT-4o-mini. This is a $16.7\times$ cost reduction, or 94%, when GPT-4o-mini replaces GPT-4o. Because the path-prompt configurations share the same retrieved candidates, evidence paths, and embedding stage, this calculation isolates the LLM validator cost and scales linearly with average prompt length. The simple-prompt diagnostic omits the evidence-path field and is therefore a lower-context comparison for assessing the added value of structural paths.

Execution environment and external services. All deterministic graph construction and Stage II retrieval experiments were run on a MacBook Air workstation with an Apple M3 chip, 8 CPU cores, and 16 GB memory, using macOS/Darwin 25.3.0 and Python 3.12.7. These retrieval-side computations are CPU-based and do not require a local GPU. The graph construction code uses spaCy for requirement-side entity and phrase extraction and tree-sitter for lightweight static parsing of code artifacts. The embedding and validation components use external OpenAI API calls with the model identifiers text-embedding-3-large, GPT-4o, and GPT-4o-mini. Validator calls set temperature to 0 for deterministic decoding; no top- p or maximum-output-token override is used, and other decoding parameters follow the API defaults. Because embeddings and LLM validation depend on external API availability, network latency, and provider pricing, we report cached-embedding retrieval time separately from validator-token cost.

3.4.4. Baselines

We compare DSDR with three baseline families: traditional information retrieval models, established traceability techniques from prior work, and LLM+RAG traceability frameworks that match our problem setting. Unless stated otherwise, all methods use the same data splits, artifact granularity, and evaluation setup; the comparison therefore focuses on retrieval and linking behavior.

We do not include supervised heterogeneous graph-learning models, such as HGNNLINK, as direct table baselines because their reported setting requires labeled trace links and project-specific message-passing training, whereas DSDR is evaluated under a label-free zero-shot RC-TLR setting. We therefore discuss heterogeneous GNN-based TLR in Related Work and scope the empirical comparison to the evaluated retrieve-then-validate setting and the baselines available under the shared benchmark protocol. A direct comparison with supervised heterogeneous GNN models under matched artifacts, labels, and training protocols is left for future work.

NOLLM is included as a retrieval-only setting rather than as an operational linker. The baseline terminates the pipeline after retrieval: for each requirement, it returns the top K code classes and applies no filtering or link decision, treating every retrieved candidate

as a positive link. With K fixed, this design measures the retriever's attainable recall and separates candidate discovery from the subsequent link-decision stage.

For requirement-to-code traceability, LiSSA [19] is the primary pipeline-level comparator because it uses the same retrieve-then-validate decomposition. LiSSA is a general-purpose TLR framework that supports multiple artifact types: IR-based retrieval narrows the candidate space and an LLM consumes the retrieved context to generate or decide trace links. We run the official LiSSA implementation and report end-to-end Precision, Recall, F_1 , and F_2 . For clarity, LiSSA is restricted to the flat artifact text and retrieved text context used by its released pipeline. It is not given DSDR's hierarchical requirement-sentence-entity and class-method-identifier graphs, entity-identifier semantic bridge, gated PPR scores, or diffusion-derived evidence paths. Retrofitting those structures into LiSSA would create a hybrid method rather than evaluate LiSSA as released. Thus, the LiSSA-DSDR comparison measures the effect of moving from a flat-text RAG retriever to the full graph-based DSDR pipeline; it does not by itself isolate the graph representation and diffusion algorithm as separate causal factors. We use the released LiSSA settings for the number of retrieved candidates, prompting strategy, and LLM inference parameters.

We also evaluate two established domain methods, FTLR [39] and COMET [40]. Results follow the authors' public implementations when available and otherwise use paper-recommended default settings to keep the comparison reproducible. Since threshold choice can dominate outcomes, we additionally report tuned-optimal variants following the LiSSA reporting convention, denoted as FTLR_{OPT} and $\text{COMET}_{\text{OPT}}$. For each project, we sweep the relevant threshold or method switch and select the setting that maximizes F_1 . These OPT variants are optimized reference points for the corresponding baseline families rather than operational zero-shot settings. These rows set a stronger reference point for DSDR, but they are not intended as a complete ranking across all RC-TLR methods: DSDR uses one fixed zero-shot configuration, while the OPT baselines show the optimized within-project setting available to those methods. Following the LiSSA evaluation [19], we report $\text{COMET}_{\text{OPT}}$ for SMOS, ETour, and iTRUST. For DRONOLGY, LiSSA treats COMET as outside supported scope; accordingly, only the two DRONOLGY entries for COMET are marked unavailable.

We further include classic text-matching baselines, VSM and LSI, to anchor performance against traditional IR. These models produce ranked code-class candidates rather than explicit link sets, so we evaluate the top K retrieved candidates and sweep K on each dataset to obtain the optimized configuration under this sweep, reported as VSM_{OPT} and LSI_{OPT} .

4. Results

4.1. Main Results

Main setup. Table 2 reports results for DSDR and the evaluated baselines on SMOS, ETour, iTRUST, DRONOLGY (RE), and DRONOLGY (DD). The table includes optimized traditional IR baselines, namely VSM_{OPT} , LSI_{OPT} , $\text{COMET}_{\text{OPT}}$, and FTLR_{OPT} , as reference points under their reported tuning conventions. The closest pipeline-level comparison is the RAG-based baseline LiSSA, evaluated under NOLLM and GPT-4o. For DSDR, NOLLM isolates retrieval behavior. The GPT-4o-mini simple-prompt row removes the structural-evidence field while using the same retrieved Top-20 candidates and the same validator model, whereas the GPT-4o-mini path-prompt row restores the extracted structural evidence paths. This pair is the Stage III evidence-path prompt ablation: it holds retrieval output and validator capacity fixed and tests whether exposing graph-derived evidence paths changes the final link decisions. The GPT-4o row then reports the higher-capacity validator using the main path-prompt setting. The table characterizes the operating points

of the evaluated methods rather than claiming overall method dominance. For readability, Table 2 uses boldface for the best value in each metric column and underlining for the second-best available value. Table 3 follows the LiSSA reporting format: it shows the p -values of the Wilcoxon signed-rank test (two-sided) for F_1/F_2 -scores against DSDR (GPT-4o). If DSDR (GPT-4o) is better than a baseline, we mark it with $\uparrow$; otherwise, we mark it with $\downarrow$. Each cell is reported as p_{F_1}/p_{F_2} with p -values rounded to two decimals, and significant entries at $\alpha = 0.05$ are bold. The Avg. column summarizes the unweighted dataset average, and w. Avg. weights datasets by the number of requirements, following the same average/weighted-average convention used in the LiSSA comparison. Dashes indicate unavailable comparisons, including entries outside supported scope or aggregate columns that cannot be constructed over all five tasks. The simple-prompt row is included as a prompt-ablation diagnostic in the main table and is not included in the pairwise significance table. The tests report raw p -values without multiple-comparison correction.

Table 2. Main results on benchmark datasets. Bold values indicate the best result in each column, and underlined values indicate the second-best result among available entries.

Approach	SMOS				eTour				iTrust				Dronology (RE)				Dronology (DD)			
	P	R	F ₁	F ₂	P	R	F ₁	F ₂	P	R	F ₁	F ₂	P	R	F ₁	F ₂	P	R	F ₁	F ₂
VSM _{OPT}	0.430	0.414	0.422	0.417	0.557	0.427	0.483	0.448	0.208	0.227	0.217	0.223	0.844	0.087	0.158	0.106	0.846	0.071	0.131	0.087
LSI _{OPT}	0.415	0.430	0.422	0.427	0.452	0.453	0.453	0.453	0.251	0.255	0.253	0.254	<u>0.333</u>	0.107	0.162	0.124	<u>0.757</u>	0.074	0.135	0.090
COMET _{OPT}	0.195	<u>0.572</u>	0.291	0.413	0.410	0.468	0.437	0.455	0.361	0.231	<u>0.282</u>	0.249	—	—	—	—	—	—	—	—
FTLR _{OPT}	0.314	0.588	0.409	0.501	<u>0.505</u>	0.597	0.548	0.576	0.234	0.241	0.238	0.240	0.184	0.170	0.177	0.173	0.140	0.147	0.144	0.146
LiSSA (NoLLM)	0.325	0.418	0.366	0.395	0.216	<u>0.815</u>	0.342	0.525	0.058	0.531	0.105	0.202	0.128	<u>0.420</u>	0.196	0.288	0.085	<u>0.482</u>	0.144	0.249
LiSSA (GPT-4o)	<u>0.590</u>	0.195	0.294	0.226	0.409	0.734	0.526	0.633	0.199	0.451	0.276	<u>0.360</u>	0.226	<u>0.344</u>	<u>0.273</u>	0.311	0.177	0.380	0.241	0.309
DSDR (NoLLM)	0.387	0.497	0.435	<u>0.470</u>	0.231	0.870	0.365	0.560	0.085	0.776	0.153	0.296	0.136	0.446	0.208	0.306	0.089	0.506	0.151	0.261
DSDR (GPT-4o-mini, simple)	0.556	0.272	0.365	0.303	0.353	0.643	0.456	0.552	0.144	<u>0.584</u>	0.231	0.362	0.152	0.375	0.216	0.290	0.142	0.287	0.190	0.238
DSDR (GPT-4o-mini, paths)	0.452	0.412	<u>0.431</u>	0.419	0.368	0.721	0.487	<u>0.605</u>	0.171	0.462	0.250	0.345	0.161	0.412	0.232	<u>0.314</u>	0.118	0.434	0.186	<u>0.283</u>
DSDR (GPT-4o)	0.663	0.250	0.363	0.286	0.478	0.610	<u>0.536</u>	0.578	<u>0.337</u>	0.357	0.347	0.353	0.253	0.312	0.278	0.339	0.220	0.263	<u>0.240</u>	0.253

Table 3. p -values of the Wilcoxon signed-rank test (two-sided) for F_1/F_2 -scores of requirement-to-code TLR with DSDR (GPT-4o) and Top-20 retrieval. Bold p -values indicate statistical significance at $\alpha = 0.05$; arrows show whether DSDR (GPT-4o) has higher or lower F_1/F_2 than the compared approach.

Approach	SMOS	eTOUR	iTRUST	DR RE	DR DD	Avg.	w. Avg.
VSM _{OPT}	0.03 ↓/ 0.03 ↓	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑
LSI _{OPT}	0.03 ↓/ 0.03 ↓	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑
COMET _{OPT}	0.03 ↑/ 0.03 ↓	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	—	—	—	—
FTLR _{OPT}	0.03 ↓/ 0.03 ↓	0.31↓/0.84↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/0.06↑	0.03 ↑/ 0.03 ↑
LiSSA (NoLLM)	0.84↓/ 0.03 ↓	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/ 0.03 ↑	0.03 ↑/0.84↑	0.03 ↑/0.09↑	0.03 ↑/0.06↑
LiSSA (GPT-4o)	0.03 ↑/ 0.03 ↑	0.44↑/ 0.03 ↓	0.03 ↑/0.56↓	0.69↑/0.09↑	0.91↓/ 0.03 ↓	0.09↑/0.56↓	0.09↑/0.22↓
DSDR (NoLLM)	0.03 ↓/ 0.03 ↓	0.03 ↑/0.12↑	0.03 ↑/ 0.03 ↑	0.03 ↑/0.09↑	0.03 ↑/0.56↓	0.03 ↑/0.22↓	0.03 ↑/0.72↓
DSDR (GPT-4o-mini, paths)	0.03 ↓/ 0.03 ↓	0.03 ↑/0.09↓	0.03 ↑/0.56↑	0.03 ↑/0.12↑	0.03 ↑/0.09↓	0.06↑/0.09↓	0.03 ↑/0.12↓

Results analysis. DSDR mainly strengthens candidate exposure before validation. For recall-oriented RC-TLR, the validator can assess only links that enter the retrieved candidate set. The retrieval-only and validator rows therefore describe two coupled effects: structural diffusion determines which candidate classes are exposed, and the LLM validator selects final links from that exposed set.

The retrieval-only comparison separates candidate generation from LLM validation. DSDR obtains higher recall than LiSSA under NoLLM on all five datasets and is the top retrieval-only setting on eTOUR, iTRUST, DRONOLGY (RE), and DRONOLGY (DD) within this comparison. The largest absolute gain occurs on iTRUST, where recall increases by 0.245 over LiSSA. Because LiSSA is evaluated with flat artifact text, this comparison reflects the combined effect of moving from flat-text RAG retrieval to DSDR's graph-based retrieval pipeline. The graph representation changes the search substrate by exposing

requirement sentences, requirement entities, code methods, code identifiers, and static code relations that are hidden in flat text. The diffusion algorithm then turns that substrate into requirement-conditioned candidate scores through semantic gating, forward/backward propagation, PPR locality, and score normalization. The ablations in Section 4.2 keep the three-layer graph fixed and remove diffusion-side components, so they further separate the algorithmic contribution of SG-DSDR from the representation substrate. The effect is most useful for sparse-link datasets, where the retriever must surface a small number of ground-truth classes from a large candidate space.

Evidence-path prompt ablation. The GPT-4o-mini simple-prompt and path-prompt rows use the same retrieved candidates and validator model, so their difference reflects the effect of exposing graph-derived evidence paths to the validator after retrieval has already been fixed. This diagnostic does not turn LiSSA into a graph-context baseline; rather, it isolates the contribution of the structural-path context used by Stage III. Compared with the simple prompt, the path prompt improves F_1 on four datasets and F_2 on four datasets; the exceptions show that the benefit of structural paths still depends on project-specific retrieval and validation behavior. Thus, the evidence paths provide useful validation context, but they change the validator operating point rather than guaranteeing a uniform increase on every metric. With GPT-4o, DSDR improves precision over LiSSA with GPT-4o on every dataset, with the clearest gains on SMOS and iTRUST. This is a precision-oriented validation outcome, not uniform dominance on every metric: the LLM validator filters the broader retrieval-only candidate set into higher-confidence links, and this filtering can reduce recall on some projects. As a result, the F_1/F_2 balance varies with project density, artifact type, prompt design, and the desired precision–recall operating point. The GPT-4o-mini path-prompt setting keeps much of the retrieval coverage while using the same structural evidence format and a cheaper validator. Together with the cost calculation in Section 3.4.3, this makes GPT-4o-mini a practical option when the application prioritizes recall and low validator cost.

4.2. Ablation Study

Ablation setup. The prompt-level evidence-path ablation is reported in Table 2 because it changes the Stage III validator input rather than the Stage II retrieval scores. Table 4 uses the same best/second-best highlighting convention to make the ablation results easier to scan. Starting from the full retrieval configuration, we construct five variants: removing semantic gating, disabling backward structural traversal, disabling forward structural traversal, dropping PPR-based propagation, and omitting z-score normalization for score calibration. All variants use the same $K = 20$ retrieval setting and report P@20, R@20, F_1 @20, and the recall-oriented F_2 @20.

Table 4. Ablation results under $K = 20$ retrieval. Bold values indicate the best result in each column, and underlined values indicate the second-best result among available entries.

Approach	SMOS				eTour				iTrust				Dronology (RE)				Dronology (DD)			
	P	R	F_1	F_2	P	R	F_1	F_2	P	R	F_1	F_2	P	R	F_1	F_2	P	R	F_1	F_2
DSDR	0.387	0.497	0.435	0.470	0.231	0.870	0.365	0.560	0.085	0.776	0.153	0.296	0.136	0.446	0.208	0.306	0.089	0.505	0.151	0.261
w/o Sem Gate	0.235	0.302	0.264	0.286	0.105	0.396	0.166	0.255	0.049	0.448	0.088	0.170	0.038	0.126	0.059	0.086	0.020	0.112	0.033	0.058
w/o Backward	<u>0.359</u>	<u>0.461</u>	<u>0.404</u>	<u>0.436</u>	<u>0.221</u>	<u>0.831</u>	<u>0.349</u>	<u>0.535</u>	<u>0.082</u>	<u>0.748</u>	<u>0.147</u>	<u>0.285</u>	<u>0.124</u>	<u>0.408</u>	<u>0.190</u>	<u>0.280</u>	<u>0.083</u>	<u>0.474</u>	<u>0.141</u>	<u>0.244</u>
w/o Forward	0.310	0.398	0.348	0.377	0.220	0.825	0.347	0.533	0.053	0.573	0.113	0.219	0.113	0.371	0.173	0.255	0.075	0.428	0.127	0.220
w/o PPR	0.346	0.443	0.388	0.419	0.217	0.818	0.343	0.526	0.054	0.497	0.098	0.188	0.123	0.406	0.189	0.278	0.080	0.457	0.136	0.235
w/o z-score	0.275	0.352	0.309	0.333	0.191	0.721	0.302	0.464	0.077	0.710	0.140	0.269	0.111	0.365	0.170	0.251	0.078	0.447	0.133	0.230

Results analysis. The full DSDR configuration is higher than every ablated variant on all reported retrieval metrics. The gain comes from the interaction of semantic gating, bidirectional structure, propagation, and calibration rather than from a single isolated step.

Semantic gating contributes the largest margin: retaining it in the full model raises recall from 0.396 to 0.870 on ETOUR and from 0.126 to 0.446 on DRONOLGY (RE) relative to the ungated variant. This fits the design motivation: structural propagation works best when requirement-specific semantic evidence selects the relevant neighborhood.

The directional components contribute different evidence. Forward traversal provides the larger contribution, so requirement-to-code diffusion is the main driver of candidate discovery. Backward traversal still improves every dataset, which means requirement-side support adds complementary evidence rather than duplicating the forward score. PPR provides multi-hop relevance propagation, most clearly on ITRUST, where the full model raises recall from 0.497 to 0.776 relative to the no-PPR variant. z-score normalization also helps across datasets because directional scores need calibration before fusion. These ablations keep the three-layer graph representation fixed and remove algorithmic components of SG-DSDR; they therefore clarify the impact of gated diffusion, directional propagation, PPR, and normalization after the graph substrate has been constructed. The ablation matches the intended decomposition of SG-DSDR: semantic gating selects requirement-relevant neighborhoods, dual-path diffusion broadens evidence, and score normalization makes heterogeneous signals comparable.

4.3. Parameter Sensitivity

Experimental setup. We examine the diagnostic behavior of the fixed parameters by varying one parameter at a time. For the forward–backward fusion weight, we sweep $\beta \in \{0.0, 0.1, \dots, 1.0\}$; $\beta = 1$ reduces the score to the forward structural signal, whereas $\beta = 0$ uses only backward requirement support. For PPR locality, we sweep the restart probability $\alpha \in \{0.0, 0.1, \dots, 1.0\}$. For candidate-list size, we evaluate $K \in \{5, 10, 20, 50\}$. Unless varied, all parameters remain at their default values, and each setting reports average Precision, Recall, F_1 , and F_2 in the retrieval-only stage. Figures 3–5 report the sweeps as dataset-level panels rather than aggregate curves. Each panel corresponds to one benchmark dataset, and each marker is the directly measured retrieval score at the corresponding parameter setting; the connecting line segments are visual guides for comparing neighboring measured points. We report these sweeps as checks on one global parameter setting, not as dataset-specific tuning. We report the bridge-threshold sweep for τ in Appendix A because it directly controls soft-alignment coverage.

Results analysis. The β curves in Figure 3 peak in an intermediate range rather than at either endpoint. This behavior is consistent with the ablation results: forward diffusion drives candidate discovery, while backward requirement support supplies complementary evidence. The dataset-level panels show different preferred mixtures: SMOS peaks near $\beta = 0.4$, ETOUR peaks near $\beta = 0.1$, ITRUST and DRONOLGY (RE) peak near $\beta = 0.7$, and DRONOLGY (DD) peaks at $\beta = 0.5$. Thus, no single endpoint dominates across datasets, and the fixed $\beta = 0.5$ setting is a stable mixed-evidence compromise rather than a dataset-wise tuned optimum. We keep the global default $\beta = 0.5$ as a balanced fixed setting for the main experiments.

The α curves in Figure 4 show the locality trade-off. Smaller restart probabilities permit broader propagation over structural neighbors, whereas larger values keep probability mass closer to the seed nodes. The preferred measured point is dataset-dependent: ITRUST is best at $\alpha = 0.3$, SMOS and DRONOLGY (RE) prefer $\alpha = 0.6$, DRONOLGY (DD) prefers $\alpha = 0.7$, and ETOUR is strongest under broader propagation. These differences indicate that α controls graph-locality behavior rather than producing a universal monotonic trend. Moderate values such as $\alpha = 0.2$ or $\alpha = 0.3$ balance locality and propagation, so we use $\alpha = 0.3$ as the fixed setting.

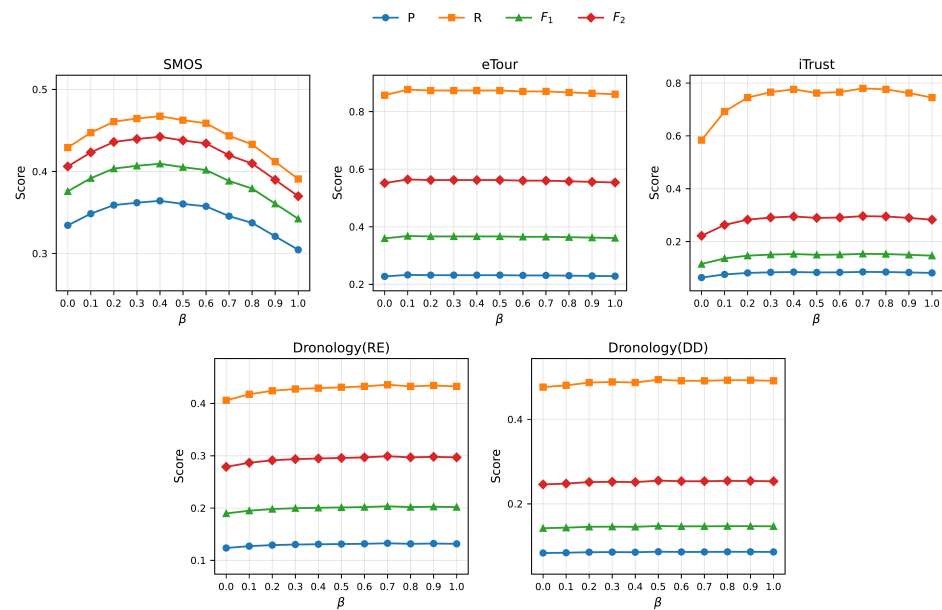


Figure 3. Effect of the forward–backward fusion weight β on retrieval performance. Each panel corresponds to one benchmark dataset. Markers are directly measured retrieval scores, and line segments are visual guides.

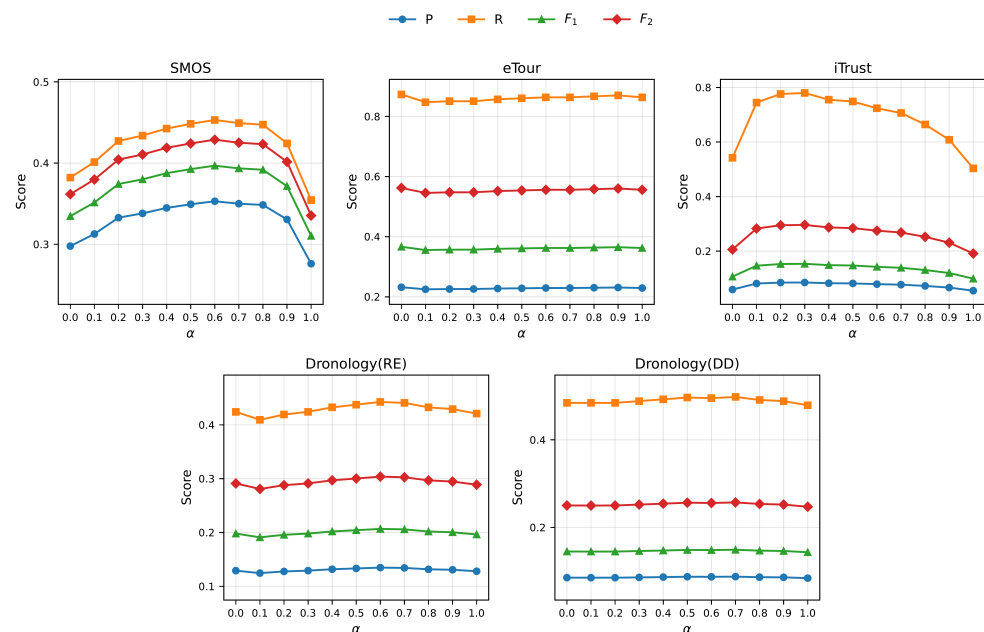


Figure 4. Effect of the PPR restart probability α on retrieval performance. Each panel corresponds to one benchmark dataset. Markers are directly measured retrieval scores, and line segments are visual guides.

The Top- K curves in Figure 5 indicate that candidate coverage improves as the candidate list grows, but the incremental gain decreases after $K = 20$. The panels also show the precision–recall trade-off separately by dataset. SMOS continues to gain in F_2 at larger K because recall dominates, whereas ETour, iTrust, and the two DRONOLGY tasks reach their best or near-best F_2 around $K = 10$ – 20 and then lose precision as K expands. Moving from $K = 10$ to $K = 20$ still gives a clear recall gain, whereas moving from $K = 20$ to $K = 50$ adds fewer positives while increasing the validator workload. We therefore use $K = 20$ in the main experiments.

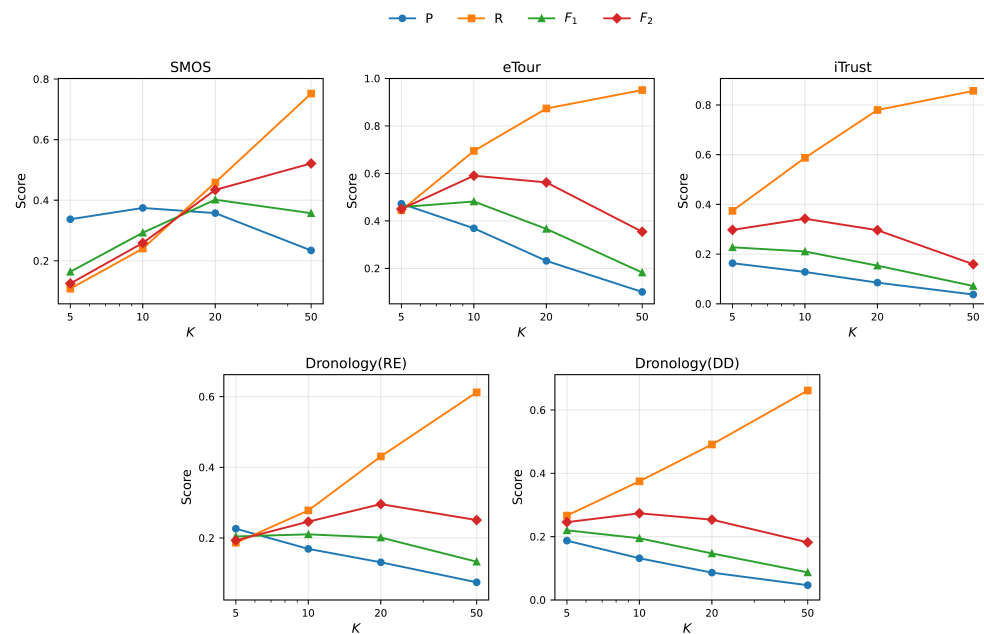


Figure 5. Effect of Top-K on retrieval-only performance. Each panel corresponds to one benchmark dataset. Markers are directly measured retrieval scores, and line segments are visual guides.

We also examined the bridge threshold τ because it controls the number of entity–identifier edges before diffusion. The sweep in Appendix A indicates that the lower-threshold region gives stronger retrieval curves, while increasing τ generally reduces bridge coverage and weakens candidate exposure. This does not mean that all extra low-threshold matches are treated as reliable candidates. Instead, the semantic gate filters the expanded bridge set by requirement relevance: useful activations can pass into structural diffusion, whereas weakly related activations receive less probability mass. This pattern revises our interpretation of τ : it should not be presented as a stable default justified by label-free operation alone. Instead, τ determines a practical coverage–sparsity trade-off, where lower thresholds favor retrieval quality through broader gated coverage and higher thresholds favor stronger pruning.

4.4. Case Study

Case setup. The case study checks whether the ranking changes in the quantitative results correspond to concrete structural evidence. Figure 6 presents two requirements from iTRUST and compares the LISSA (NOLLM) retrieval ranking with the ranking after SG-DSDR. The first example is a positive recovery case. The second is a cautionary negative case: structural reinforcement can overemphasize a convergent class even when the relevant classes remain visible.

Results analysis. For requirement UC4S1, LISSA (NOLLM) ranks the ground-truth implementation 62nd, outside the top 20 candidates. SG-DSDR changes this outcome by using the structural neighborhood around related action classes. The top 20 list contains candidates such as EditPHRAAction, EditPatientAction, and ViewMyRecordsAction, which directly or indirectly invoke PatientDA0. Semantic-gated propagation lets these convergent signals reinforce PatientDA0, raising its aggregated score to rank 2. This positive case shows SG-DSDR recovering an implementation with low surface similarity but strong code-structure support.

For UC2S1, the same NOLLM ranking already places the two ground-truth classes AddHCPAction and PersonnelDA0 at ranks 2 and 6. After structural retrieval, PersonnelDA0 rises to rank 1, while AddHCPAction remains within the top 10. The retrieved structure

explains this shift: several high-ranked candidates connect to `PersonnelDAO`, so the class receives repeated support during propagation. This negative case marks a limitation of structural reinforcement. When many paths converge on a data-access or hub-like class, SG-DSDR can overemphasize that class relative to a more requirement-specific action class. Row-wise z-score calibration and the semantic gate reduce this effect by keeping reinforcement requirement-conditioned, but they do not remove it completely.

Requirement UC4S1: "A patient or personal health representative may enter or edit their own demographic informatio....."		
① Semantic matching	② Structure verification	③ Final result
Rank 1: SecurityQA (Score 0.4158)		Rank 1: SecurityQA (Score 3.0856)
Rank 2: SetSecurityQuestionAction (Score 0.4071)	EditPHRAction -> Calls -> PatientDAO	Rank 2: PatientDAO (Score 2.8996)
Rank 3: PatientBean (Score 0.4037)	EditPatientAction -> Calls -> PatientDAO	Rank 3: PersonnelDAO (Score 2.8254)
Rank 4: EditPHRAction (Score 0.3866)	ViewMyRecordsAction -> Calls -> PatientDAO	Rank 4: TransactionDAO (Score 2.6372)
Rank 5: EditPatientAction (Score 0.3592)	EditRepresentativesAction -> Calls -> PatientDAO	Rank 5: SecurityQAValidator (Score 2.5189)
.....		
Rank 8: ViewMyRecordsAction (Score 0.3399)		
.....		
Rank 62: PatientDAO (Score 0.2483)		
Requirement UC2S1: "An administrator enters a LHCP, ER, or public health agent as a user of iTrust MedicalRecords system, initially only the name and email are provided....."		
① Semantic matching	② Structure verification	③ Final result
Rank 1: ManageHospitalAssignmentsAction (Score 0.5060)		Rank 1: PersonnelDAO (Score 6.5889)
Rank 2: AddHCPAction (Score 0.5010)	AddHCPAction -> Calls -> PatientDAO	Rank 2: TransactionDAO (Score 4.9210)
Rank 3: AddERespAction (Score 0.4932)	ManageHospitalAssignmentsAction->Calls-> PatientDAO	Rank 3: ErrorList (Score 4.7384)
Rank 4: EditPHRAction (Score 0.4833)	AddERespAction -> Calls -> PatientDAO	Rank 4: EmailUtil (Score 4.4171)
Rank 5: DeclareHCPAction (Score 0.4766)		
Rank 6: PersonnelDAO (Score 0.4720)		Rank 10: AddHCPAction (Score 3.1037)
.....		

Figure 6. iTrust case study showing one positive recovery case and one cautionary negative case for requirements UC4S1 and UC2S1. Red labels denote ground-truth implementation classes.

5. Discussion

The main lesson for GraphRAG-based RC-TLR is that graph structure should organize evidence, not only expand candidate coverage. In DSDR, the same requirement–code graph supplies semantic bridges, diffusion scores, and backtracked evidence paths. The validator therefore depends less on isolated textual similarity and more on inspectable structural support. The ablation and sensitivity analyses point to several interacting components: semantic gating selects requirement-relevant neighborhoods, dual-path propagation broadens structural evidence, PPR controls locality, and score normalization makes the directional signals comparable.

5.1. Interpretation and Practical Implications

Compared with retrieval pipelines that rely mainly on artifact text, such as LiSSA, DSDR changes the retrieval unit from isolated textual fragments to paths over a heterogeneous requirement–code graph. LLM-based structural validation is bounded by the candidate set exposed by retrieval; omitted ground-truth classes do not reach the link-decision stage. By retrieving through semantic bridges and local code structure, DSDR exposes candidates that have weak surface similarity but stronger structural support.

HGNNLINK and related graph-learning approaches show why heterogeneous software graphs are useful, but they usually rely on learned message-passing parameters and supervised link-prediction training. DSDR uses graph structure as a zero-shot retrieval substrate instead. We treat it as a retrieval-and-validation design rather than as a new neural graph model: it combines cross-modal semantic bridging, gated diffusion, backward requirement support, and path-based evidence construction within a zero-shot RC-TLR pipeline.

The case study also clarifies how to read high-degree classes. A high-degree class becomes informative only when semantically gated paths from the current requirement converge on it. Row-wise z-score normalization and the requirement-specific semantic gate reduce global centrality effects by tying structural reinforcement to the current requirement

context. Appendix A quantifies this behavior by comparing Top-20 candidate in-degree before and after semantic gating. A developer-facing traceability tool should expose the retrieved evidence paths so that users can inspect how each high-scoring class is supported.

The LLM validator is a decision module conditioned on retrieved evidence, not an unconditioned trace-link classifier. In our experiments, GPT-4o favors precision, whereas GPT-4o-mini is cheaper and gives a different precision–recall profile. The retrieval component supplies structurally supported candidates and paths; the final link set still depends on validator choice, prompt stability, and deployment cost in the target environment.

5.2. Limitations and Threats to Validity

The evaluation reflects the benchmark setting used in this study. The five tasks are dominated by Java projects of moderate size, so the reported gains are most directly supported for projects with similar artifact types, label granularity, and code organization. Evaluation on substantially larger industrial codebases remains future work. The structural graph captures containment, explicit static calls, and inheritance, giving a stable graph definition for controlled analysis. The current study does not incorporate dynamic dependency information, so the retrieval results should be interpreted as static-graph results. Because dynamic dependencies are outside the current static graph, their possible effect on retrieval-stage false negatives remains an open issue. Extending DSDR with dynamic dependency analysis and evaluating its influence on retrieval coverage are important directions for future work.

The benchmark labels set class-level output granularity. Although DSDR uses method and identifier nodes internally, the final output is a class-level ranking to match the available labels. This granularity can hide method-level localization quality and can make class-level scores less precise for large or multi-responsibility classes. In settings with method-level labels or developer-facing localization requirements, more selective aggregation, such as max pooling, top- m pooling, or attention over method evidence, can give a finer-grained ranking.

Several calibration choices are fixed in the current evaluation setup. The bridge threshold τ , PPR restart probabilities, number of retained candidates K , and fusion weight β are shared across all datasets, and no dataset-specific validation labels are used to tune them. This keeps the evaluation consistent with the zero-shot setting, but project-specific vocabulary, identifier style, and graph density can shift the preferred operating point. The threshold sweep in Appendix A shows that lower soft-alignment thresholds can improve retrieval metrics by increasing bridge coverage, so τ should be calibrated as an operating parameter when deployment labels, runtime budgets, or graph-density constraints are available. Practical deployments can use adaptive calibration based on the distribution of entity–identifier similarities, graph sparsity, or degree statistics.

The embedding model is also fixed to the LISSA configuration. We use text-embedding-3-large throughout the experiments to avoid changing the semantic encoder when comparing DSDR with the main RAG-style baseline. Other embedding models, including open-source or code-specialized encoders, may change absolute scores. The claims in this paper therefore concern structural diffusion and evidence-guided validation under the embedding setting used for the LISSA comparison, not a comparison among embedding model families.

The LLM validator is a deployment component whose behavior depends on the selected model, prompt template, and cost budget. In the evaluated configurations, GPT-4o and GPT-4o-mini expose different precision–recall trade-offs. The path prompt uses the supportive evidence paths retrieved for the queried pair and does not construct opposing paths for adversarial or debate-style verification. Such contradictory paths cannot be

obtained by simply reusing the current graph-retrieval output; they require an additional counter-evidence component that searches for alternative requirement associations or competing structural explanations. A future extension could combine supportive and opposing paths in an LLM debate-style verification process before the final trace decision. This design may help reduce false positives from plausible but non-specific links, but it is outside the scope of the current evaluation. Validator selection and prompt design are therefore operating choices rather than fixed properties of the retriever.

6. Conclusions

We presented DSDR, a GraphRAG-based dual-path structural diffusion retrieval framework for zero-shot requirement-code traceability link recovery. DSDR represents requirements and code as aligned three-layer heterogeneous graphs, connects requirement entities to code identifiers through a soft semantic bridge, and ranks candidate classes by fusing semantic-gated forward diffusion with backward requirement support. It extracts complete evidence paths from the diffusion states and uses those paths to ground LLM-based structural validation.

Across five benchmark tasks, retrieval-only DSDR exposes more ground-truth classes before LLM-based structural validation, improves recall over the LiSSA retrieval baseline on all five tasks, and is the top retrieval-only setting on four tasks in this comparison. After validation with GPT-4o, DSDR produces a trace-link set with higher precision than the LiSSA GPT-4o pipeline, while recall-oriented metrics depend on the project and validator operating point. The ablation results show that semantic gating, dual-path retrieval, PPR-based propagation, and z-score normalization each contribute to retrieval quality. The evidence in these experiments supports structural retrieval for zero-shot RC-TLR and lets the LLM validator operate at an explicit precision–recall point.

DSDR can be extended with richer program analyses, finer-grained labels, adaptive calibration, and alternative validator configurations. The graph representation gives direct entry points for these changes: additional dependency relations can be added as graph edges, method-level labels can be used before class aggregation, and validator choices can be adjusted to match deployment cost and precision–recall preferences.

Author Contributions: Conceptualization, Z.Z. and X.X.; Methodology, Z.Z.; Software, Z.Z.; Validation, Z.Z.; Formal analysis, Z.Z.; Investigation, Z.Z.; Data curation, Z.Z.; Writing—original draft preparation, Z.Z.; Writing—review and editing, Z.Z. and X.X.; Visualization, Z.Z.; Supervision, X.X.; Project administration, X.X. All authors have read and agreed to the published version of the manuscript.

Funding: This research is supported by the “Pioneer” and “Leading Goose” R&D Program of Zhejiang under Grant No. 2024C01020.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets analyzed in this study are third-party benchmark datasets. The SMOS, eTOUR, and iTRUST artifacts are available from the replication package released by Hey et al. [13]. The DRONOLGY artifacts are available from the dataset introduced by Cleland-Huang et al. [38]. The ground-truth requirement–code links used for evaluation are available from the LiSSA replication materials [19]. No new benchmark dataset was generated. The experimental code, processed intermediate data, and code artifacts used in this study are available from the corresponding author upon reasonable request.

Acknowledgments: The authors thank the maintainers of the public traceability benchmark datasets and replication packages used in this study.

Conflicts of Interest: The authors declare no conflicts of interest.

Appendix A. Diagnostic Sensitivity, Retrieval-Time, and Hub Analyses

Figure A1 reports the soft-alignment threshold sensitivity analysis requested for the entity–identifier bridge. The sweep uses a fine-grained interval for τ and keeps all other retrieval settings fixed. To keep the analysis focused on retrieval behavior, the figure reports only Precision, Recall, F_1 , and F_2 .

The curves show that lower-threshold settings generally produce stronger retrieval performance because more entity–identifier bridges remain available for structural diffusion. Although these settings introduce more initial bridge matches, the semantic gate filters the expanded candidate space by requirement relevance, allowing useful candidates to remain active while suppressing weakly related activations. As τ increases, soft-alignment coverage is pruned more aggressively, which can weaken candidate exposure before LLM validation. We therefore interpret τ as a coverage–sparsity operating parameter, rather than using this sweep to claim that one fixed threshold is universally stable across projects.

Cached-embedding retrieval time. Table A1 reports wall-clock time-to-retrieve for the Stage II retriever under the main Top-20 setting. The measurements use cached embeddings and exclude LLM validation calls, so they isolate the deterministic retrieval side of the pipeline. The timing measurements were obtained on the local CPU workstation described in Section 3.4.3; they exclude embedding-generation latency, OpenAI API network latency, and LLM validation time. “Graph build” measures construction of the static structural matrices, “Stage II” measures bridge construction, semantic scoring, and the forward/backward gated PPR runs used to produce the ranked candidate lists, and “Total” is the sum of these two retrieval-side components.

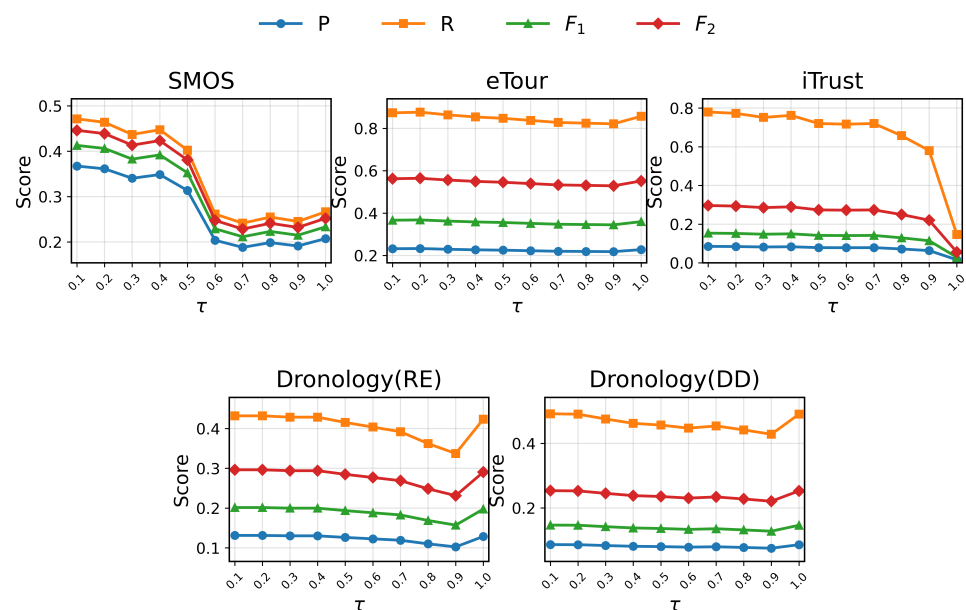


Figure A1. Soft-alignment threshold sensitivity by dataset under Top-20 retrieval.

Table A1. Cached-embedding time-to-retrieve under the main Top-20 retrieval setting.

Dataset	Req.	Code	Graph Build (s)	Stage II (s)	Total (s)	ms/Req.
SMOS	67	99	2.72	0.49	3.21	47.9
ETOUR	58	116	3.38	0.91	4.29	74.0
ITRUST	131	226	3.42	1.90	5.32	40.6
DRONOLGY (RE)	99	423	2.43	1.66	4.08	41.2
DRONOLGY (DD)	211	423	3.32	3.10	6.42	30.4

The measurements show that, after embeddings are cached, Stage II retrieval remains within seconds on all five benchmark tasks. The PPR component is reused to produce one ranked candidate list per requirement, and these ranked lists are shared by both validator configurations. This timing behavior supports the intended deployment pattern: graph construction and Stage II diffusion are deterministic preprocessing/retrieval steps, while the main variable cost comes from the downstream validator calls and the chosen candidate budget K . Together with the validator-token estimate in Section 3.4.3, these measurements separate deterministic retrieval time from the LLM validation cost, giving a simple runtime and cost breakdown for the evaluated pipeline.

Hub-artifact in-degree analysis. To quantify hub-artifact mitigation, we compare candidate in-degree before and after semantic gating. For each dataset, the in-degree of a code class is the number of requirements for which that class appears in the Top-20 retrieved candidate list. The ungated setting removes semantic gating while keeping the same graph, bridge, PPR, and ranking pipeline. Table A2 reports the maximum Top-20 in-degree and the mean in-degree of the five highest-in-degree ungated classes, then measures how much those same five classes are reduced after semantic gating.

The reduction is strongest on the two DRONOLGY tasks and on ETour, where ungated retrieval repeatedly promotes broadly connected infrastructure-like classes. The effect is weaker on SMOS and ITRUST; some high-degree classes remain in many candidate lists because they are still semantically close to many requirements under the evaluated representation. Thus, semantic gating mitigates hub dominance by making diffusion requirement-conditioned, but it does not forcibly remove classes that continue to receive requirement-specific semantic support.

Table A2. Top-20 candidate in-degree before and after semantic gating.

Dataset	Req.	Max In-Degree		Top-5 Ungated Hubs		
		Ungated	Gated	Ungated Mean	Gated Mean	Reduction
SMOS	67	67	67	67.0	66.2	1.2%
ETour	58	35	23	29.4	15.6	46.9%
ITRUST	131	131	131	131.0	111.2	15.1%
DRONOLGY (RE)	99	98	27	94.4	10.2	89.2%
DRONOLGY (DD)	211	211	76	183.2	31.2	83.0%

References

1. Gotel, O.C.; Finkelstein, C. An analysis of the requirements traceability problem. In Proceedings of the IEEE International Conference on Requirements Engineering, Colorado Springs, CO, USA, 18–22 April 1994; pp. 94–101.
2. Wang, B.; Peng, R.; Li, Y.; Lai, H.; Wang, Z. Requirements traceability technologies and technology transfer decision support: A systematic review. *J. Syst. Softw.* **2018**, *146*, 59–79. [\[CrossRef\]](#)
3. Cleland-Huang, J.; Gotel, O.C.; Huffman Hayes, J.; Mäder, P.; Zisman, A. Software traceability: Trends and future directions. In *Future of Software Engineering Proceedings*; Association for Computing Machinery: New York, NY, USA, 2014; pp. 55–69.
4. Cleland-Huang, J.; Gotel, O.; Zisman, A. *Software and Systems Traceability*; Springer: Berlin/Heidelberg, Germany, 2012.
5. Hayes, J.H.; Dekhtyar, A.; Sundaram, S.K. Advancing candidate link generation for requirements tracing: The study of methods. *IEEE Trans. Softw. Eng.* **2006**, *32*, 4–19. [\[CrossRef\]](#)
6. Antoniol, G.; Canfora, G.; Casazza, G.; De Lucia, A.; Merlo, E. Recovering traceability links between code and documentation. *IEEE Trans. Softw. Eng.* **2002**, *28*, 970–983. [\[CrossRef\]](#)
7. Keenan, E.; Czauderna, A.; Leach, G.; Cleland-Huang, J.; Shin, Y.; Moritz, E.; Gethers, M.; Poshyvanyk, D.; Maletic, J.; Hayes, J.H.; et al. Tracelab: An experimental workbench for equipping researchers to innovate, synthesize, and comparatively evaluate traceability solutions. In Proceedings of the 2012 34th International Conference on Software Engineering (ICSE), Zurich, Switzerland, 2–9 June 2012; pp. 1375–1378.
8. Marcus, A.; Maletic, J.I. Recovering documentation-to-source-code traceability links using latent semantic indexing. In Proceedings of the 25th International Conference on Software Engineering, Portland, OR, USA, 3–10 May 2003; pp. 125–135.

9. Mahmoud, A.; Niu, N.; Xu, S. A semantic relatedness approach for traceability link recovery. In Proceedings of the 2012 20th IEEE International Conference on Program Comprehension (ICPC), Passau, Germany, 11–13 June 2012; pp. 183–192.
10. Mahmoud, A.; Niu, N. On the role of semantics in automated requirements tracing. *Requir. Eng.* **2015**, *20*, 281–300. [\[CrossRef\]](#)
11. Guo, J.; Gibiec, M.; Cleland-Huang, J. Tackling the term-mismatch problem in automated trace retrieval. *Empir. Softw. Eng.* **2017**, *22*, 1103–1142. [\[CrossRef\]](#)
12. Li, T.; Wang, S.; Lillis, D.; Yang, Z. Combining machine learning and logical reasoning to improve requirements traceability recovery. *Appl. Sci.* **2020**, *10*, 7253. [\[CrossRef\]](#)
13. Hey, T.; Chen, F.; Weigelt, S.; Tichy, W.F. Improving traceability link recovery using fine-grained requirements-to-code relations. In Proceedings of the 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), Luxembourg, 27 September–1 October 2021; pp. 12–22. [\[CrossRef\]](#)
14. Gao, H.; Kuang, H.; Assunção, W.K.; Mayr-Dorn, C.; Rong, G.; Zhang, H.; Ma, X.; Egyed, A. Triad: Automated traceability recovery based on biterm-enhanced deduction of transitive links among artifacts. In Proceedings of the IEEE/ACM 46th International Conference on Software Engineering, Lisbon, Portugal, 14–20 April 2024; pp. 1–13.
15. Gethers, M.; Oliveto, R.; Poshyvanyk, D.; De Lucia, A. On integrating orthogonal information retrieval methods to improve traceability recovery. In Proceedings of the 2011 27th IEEE International Conference on Software Maintenance (ICSM), Williamsburg, VA, USA, 25–30 September 2011; pp. 133–142.
16. Lin, J.; Liu, Y.; Zeng, Q.; Jiang, M.; Cleland-Huang, J. Traceability transformed: Generating more accurate links with pre-trained bert models. In Proceedings of the 2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE), Madrid, Spain, 25–28 May 2021; pp. 324–335.
17. Wang, B.; Zou, Z.; Liang, X.; Jin, H.; Liang, P. HGNNLink: Recovering requirements-code traceability links with text and dependency-aware heterogeneous graph neural networks. *Autom. Softw. Eng.* **2025**, *32*, 55. [\[CrossRef\]](#)
18. Ali, S.J.; Naganathan, V.; Bork, D. Establishing traceability between natural language requirements and software artifacts by combining rag and llms. In *Proceedings of the International Conference on Conceptual Modeling*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 295–314. [\[CrossRef\]](#)
19. Fuchß, D.; Hey, T.; Keim, J.; Liu, H.; Ewald, N.; Thirolf, T.; Koziol, A. LiSSA: Toward generic traceability link recovery through retrieval-augmented generation. In Proceedings of the 2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE), Ottawa, ON, Canada, 27 April–3 May 2025; pp. 1396–1408. [\[CrossRef\]](#)
20. Hey, T.; Fuchß, D.; Keim, J.; Koziol, A. Requirements Traceability Link Recovery via Retrieval-Augmented Generation. In *Proceedings of the International Working Conference on Requirements Engineering: Foundation for Software Quality*; Springer: Berlin/Heidelberg, Germany, 2025; pp. 381–397. [\[CrossRef\]](#)
21. Zhu, J.; Xiao, G.; Zheng, Z.; Sui, Y. Enhancing traceability link recovery with unlabeled data. In Proceedings of the 2022 IEEE 33rd International Symposium on Software Reliability Engineering (ISSRE), Charlotte, NC, USA, 31 October–3 November 2022; pp. 446–457.
22. Wang, B.; Zou, Z.; Wan, H.; Li, Y.; Deng, Y.; Li, X. An empirical study on the state-of-the-art methods for requirement-to-code traceability link recovery. *J. King Saud Univ.-Comput. Inf. Sci.* **2024**, *36*, 102118. [\[CrossRef\]](#)
23. Salton, G.; Wong, A.; Yang, C.S. A vector space model for automatic indexing. *Commun. ACM* **1975**, *18*, 613–620. [\[CrossRef\]](#)
24. Blei, D.M.; Ng, A.Y.; Jordan, M.I. Latent dirichlet allocation. *J. Mach. Learn. Res.* **2003**, *3*, 993–1022.
25. Asuncion, H.U.; Asuncion, A.U.; Taylor, R.N. Software traceability with topic modeling. In Proceedings of the 32nd ACM/IEEE International Conference on Software Engineering, Cape Town, South Africa, 1–8 May 2010; Volume 1, pp. 95–104.
26. Meedeniya, D.; Rubasinghe, I.; Perera, I. Traceability Establishment and Visualization of Software Artefacts in DevOps Practice: A Survey. *Int. J. Adv. Comput. Sci. Appl.* **2019**, *10*, 66–76. [\[CrossRef\]](#)
27. Dasgupta, T.; Grechanik, M.; Moritz, E.; Dit, B.; Poshyvanyk, D. Enhancing software traceability by automatically expanding corpora with relevant documentation. In Proceedings of the 2013 IEEE International Conference on Software Maintenance, Eindhoven, The Netherlands, 22–28 September 2013; pp. 320–329.
28. Gao, H.; Kuang, H.; Sun, K.; Ma, X.; Egyed, A.; Mäder, P.; Rong, G.; Shao, D.; Zhang, H. Using consensual biterns from text structures of requirements and code to improve IR-based traceability recovery. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*; ACM: New York, NY, USA, 2022; pp. 114:1–114:13. [\[CrossRef\]](#)
29. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Proceedings of the Findings of the Association for Computational Linguistics: EMNLP 2020*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2020; pp. 1536–1547. [\[CrossRef\]](#)
30. Eaddy, M.; Aho, A.V.; Antoniol, G.; Guéhéneuc, Y.G. Cerberus: Tracing requirements to source code using information retrieval, dynamic analysis, and program analysis. In Proceedings of the 2008 16th IEEE International Conference on Program Comprehension, Amsterdam, The Netherlands, 10–13 June 2008; pp. 53–62.

31. McMillan, C.; Poshyvanyk, D.; Revelle, M. Combining textual and structural analysis of software artifacts for traceability link recovery. In Proceedings of the 2009 ICSE Workshop on Traceability in Emerging Forms of Software Engineering, Vancouver, BC, Canada, 18 May 2009; pp. 41–48.
32. Nishikawa, K.; Washizaki, H.; Fukazawa, Y.; Oshima, K.; Mibe, R. Recovering transitive traceability links among software artifacts. In Proceedings of the 2015 IEEE International Conference on Software Maintenance and Evolution (ICSME), Bremen, Germany, 29 September–1 October 2015; pp. 576–580.
33. Rodriguez, A.D.; Cleland-Huang, J.; Falessi, D. Leveraging intermediate artifacts to improve automated trace link retrieval. In Proceedings of the 2021 IEEE International Conference on Software Maintenance and Evolution (ICSME), Luxembourg, 27 September–1 October 2021; pp. 81–92.
34. Zou, Z.; Wang, B.; Hu, X.; Deng, Y.; Wan, H.; Jin, H. Enhancing requirements-to-code traceability with GA-XWCoDe: Integrating XGBoost, Node2Vec, and genetic algorithms for improving model performance and stability. *J. King Saud Univ.-Comput. Inf. Sci.* **2024**, *36*, 102197. [[CrossRef](#)]
35. Rodriguez, A.D.; Dearstyne, K.R.; Cleland-Huang, J. Prompts matter: Insights and strategies for prompt engineering in automated software traceability. In Proceedings of the 2023 IEEE 31st International Requirements Engineering Conference Workshops (REW), Hannover, Germany, 4–5 September 2023; pp. 455–464.
36. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Ichter, B.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
37. Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.t.; Rocktäschel, T.; et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Adv. Neural Inf. Process. Syst.* **2020**, *33*, 9459–9474.
38. Cleland-Huang, J.; Vierhauser, M.; Bayley, S. Dronology: An incubator for cyber-physical system research. In *Proceedings of the 40th International Conference on Software Engineering: New Ideas and Emerging Results*; ACM: New York, NY, USA, 2018; pp. 109–112. [[CrossRef](#)]
39. Hey, T.; Keim, J.; Corallo, S. Requirements classification for traceability link recovery. In Proceedings of the 2024 IEEE 32nd International Requirements Engineering Conference (RE), Reykjavik, Iceland, 24–28 June 2024; pp. 155–167. [[CrossRef](#)]
40. Moran, K.; Palacio, D.N.; Bernal-Cárdenas, C.; McCrystal, D.; Poshyvanyk, D.; Shenefiel, C.; Johnson, J. Improving the effectiveness of traceability link recovery using hierarchical bayesian networks. In Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering, Seoul, Republic of Korea, 27–19 July 2020; pp. 873–885.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.